DDA3020 Machine Learning
Lecture 05 Linear Regression

Jicong Fan
School of Data Science, CUHK-SZ

02/13/2025

Outline

1. Notations, vectors, matrices
2. Functions, derivative and gradient
3. Modeling of linear regression
   - Deterministic perspective
   - Probabilistic perspective
4. Learning of linear regression
   - Analytical solution
   - Gradient descent algorithm
5. Linear regression of multiple outputs
6. Linear regression for classification
7. Variants of linear regression
   - Ridge regression
   - Polynomial regression
   - Lasso regression
   - Robust linear regression

1. Notations, vectors, matrices
2. Functions, derivative and gradient
3. Modeling of linear regression
   - Deterministic perspective
   - Probabilistic perspective
4. Learning of linear regression
   - Analytical solution
   - Gradient descent algorithm
5. Linear regression of multiple outputs
6. Linear regression for classification
7. Variants of linear regression
   - Ridge regression
   - Polynomial regression
   - Lasso regression
   - Robust linear regression

Notations

- A **set** is an unordered collection of unique elements.
- We denote a set as a calligraphic capital character, for example, $\mathcal{S}$.
- A set of numbers can be finite (include a fixed amount of values). In this case, it is denoted using accolades, for example, $\{1, 3, 18, 23, 235\}$ or $\{x_1, x_2, x_3, ..., x_d\}$.
- A set can also be infinite.

Notations

- A set can be infinite and include all values in some interval.
- If a set includes all values between a and b, including a and b, it is denoted using brackets as $[a, b]$.
- If the set does not include the values a and b, such a set is denoted using parentheses like this: (a, b).
- For example, the set [0, 1] includes such values as 0, 0.0001, 0.25, 0.784, 0.9995, and 1.0.
- A special set denoted $\mathcal{R}$ (or R, $\mathbb{R}$) includes all real numbers from minus infinity to plus infinity.

Notations

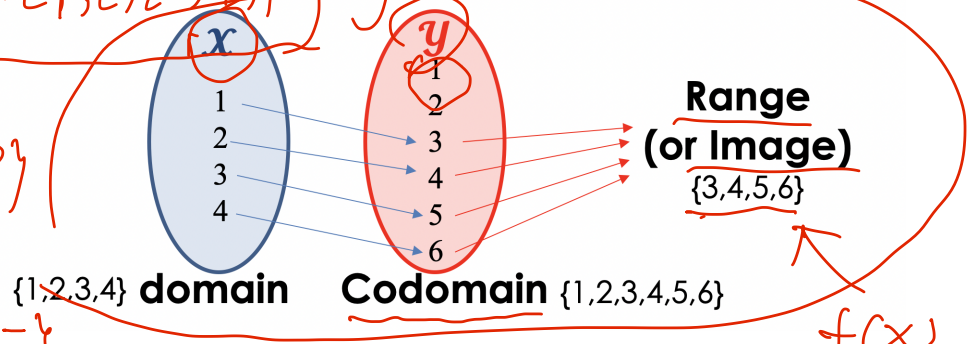
- Intersection of two sets:
  $\mathcal{S}_3 \leftarrow \mathcal{S}_1 \cap \mathcal{S}_2$
  Example: $\{1, 3, 5, 8\} \cap \{1, 8, 4\} = \{1, 8\}$

- Union of two sets:
  $\mathcal{S}_3 \leftarrow \mathcal{S}_1 \cup \mathcal{S}_2$
  Example: $\{1, 3, 5, 8\} \cup \{1, 8, 4\} = \{1, 3, 4, 5, 8\}$

1. Notations, vectors, matrices
2. Functions, derivative and gradient
3. Modeling of linear regression
   - Deterministic perspective
   - Probabilistic perspective
4. Learning of linear regression
   - Analytical solution
   - Gradient descent algorithm
5. Linear regression of multiple outputs
6. Linear regression for classification
7. Variants of linear regression
   - Ridge regression
   - Polynomial regression
   - Lasso regression
   - Robust linear regression

Functions, derivative and gradient

- A **function** is a relation that associates each element x of a set $\mathcal{X}$, the **domain** of the function, to a single element y of another set $\mathcal{Y}$, the **codomain** of the function.
- A function usually has a name. If the function is called f, this relation is denoted $y = f(x)$ (read f of x), the element x is the argument or input of the function, and y is the value of the function or the output.
- The symbol that is used for representing the input is the variable of the function (we often say that f is a function of the variable x).



Functions, derivative and gradient

- A **scalar function** can also have vector argument such as, $y = f(\mathbf{x})$, or a scalar argument ($y = f(x)$).
- A **vector function**, denoted as $\mathbf{y} = \mathbf{f}(\mathbf{x})$, is a function that returns $\mathbf{y}$, which can have either a vector argument ($\mathbf{y} = \mathbf{f}(\mathbf{x})$) or a scalar argument ($\mathbf{y} = \mathbf{f}(x)$).

Functions, derivative and gradient

Notation

- The notation $f : \mathbb{R}^d \to \mathbb{R}$ means that f is a function that maps real d-vectors to real numbers, *i.e.*, it is a scalar-valued function of d dimension vectors.
- If $\mathbf{x}$ is a d-vector, then $f(\mathbf{x})$, which is a scalar, denotes the value of the function f at $\mathbf{x}$. In the notation $f(\mathbf{x})$, $\mathbf{x}$ is referred to as the argument of the function

$$f(\mathbf{x}) = f(x_1, x_2, \ldots, x_d)$$

Functions, derivative and gradient

Linear and Affine functions

- To describe a function $f : \mathbb{R}^d \to \mathbb{R}$ we have to specify its value for any possible argument $\mathbf{x} \in \mathbb{R}^d$.
- For example, we can define a function $f : \mathbb{R}^4 \to \mathbb{R}$ by

$$f(\mathbf{x}) = x_1 + x_2 - 2x_3 - x_4$$

$f(x) = a^T x$

$f(x) = Ax$

$f(x) = AX$

Linear and Affine functions

Another example: The inner product function

- Suppose there is a d-vector. We can define a scalar-valued function f of d-vectors, given by

$$f(\mathbf{x}) = \mathbf{a}^\top \mathbf{x} = a_1 x_1 + a_2 x_2 + \ldots + a_d x_d$$

  for any d-vector $\mathbf{x}$.

- This function gives the inner product of its d-vector argument $\mathbf{x}$ with some (fixed) d-vector $\mathbf{a}$.
- We can also think of f as forming a weighted sum of the elements of $\mathbf{x}$; the elements of $\mathbf{a}$ give the weights used in forming the weighted sum.

Functions, derivative and gradient

Linear and Affine functions

- A function $f : \mathbb{R}^d \to \mathbb{R}$ is **linear** if it satisfies the following two properties:
  - **Homogeneity**: For any d-vector $\mathbf{x}$ and a scalar α, $f(\alpha \mathbf{x}) = \alpha f(\mathbf{x})$.
  - **Additivity**: For any d-vectors $\mathbf{x}$ and $\mathbf{y}$, $f(\mathbf{x} + \mathbf{y}) = f(\mathbf{x}) + f(\mathbf{y})$

  $$f(\mathbf{x}) = \mathbf{a}^\top \mathbf{x} = a_1 x_1 + a_2 x_2 + \ldots + a_d x_d$$

  for any d-vector $\mathbf{x}$.
- **Homogeneity** states that scaling the (vector) argument is the same as scaling the function value.
- **Additivity** says that adding (vector) arguments are the same as adding the function values.

Linear and Affine functions

Superposition and linearity

- The inner product function f defined before satisfies the linearity property

$$\begin{aligned} f(\alpha\mathbf{x}+\beta\mathbf{y}) &= \mathbf{a}^\top(\alpha\mathbf{x}+\beta\mathbf{y}) \\ &= \mathbf{a}^\top(\alpha\mathbf{x})+\mathbf{a}^\top(\beta\mathbf{y}) \\ &= \alpha(\mathbf{a}^\top\mathbf{x})+\beta(\mathbf{a}^\top\mathbf{y}) \\ &= \alpha f(\mathbf{x})+\beta f(\mathbf{y}) \end{aligned}$$

  for all d-vectors $\mathbf{x}, \mathbf{y}$, and all scalars α, β.

- This property is called **superposition** (which consists of homogeneity and additivity).
- A **function** that satisfies the superposition property is called **linear**

Functions, derivative and gradient

Linear and Affine functions

- If a function f is linear, superposition extends to linear combinations of any number of vectors:

$$f(\alpha_1 \mathbf{x}_1 + \ldots + \alpha_k \mathbf{x}_k) = \alpha_1 f(\mathbf{x}_1) + \ldots + \alpha_k f(\mathbf{x}_k)$$

for any d-vectors $\mathbf{x}_1$, ..., $\mathbf{x}_k$ and any scalars α_1,...,α_k

Functions, derivative and gradient

Linear and Affine functions

A function $f : \mathbb{R}^d \to \mathbb{R}$ is affine if and only if it can be expressed as $f(\mathbf{x}) = \mathbf{a}^\top \mathbf{x} + b$ for some d-vector $\mathbf{a}$ and a scalar b, which is sometimes called the offset.

$\langle A, X \rangle$

intercept

bias

Example:

$$f(\mathbf{x}) = 2.3 - 2x_1 + 1.3x_2 - x_3$$

is affine, with $b = 2.3$, $\mathbf{a} = \begin{bmatrix} -2 \\ 1.3 \\ -1 \end{bmatrix}$.

Functions, derivative and gradient

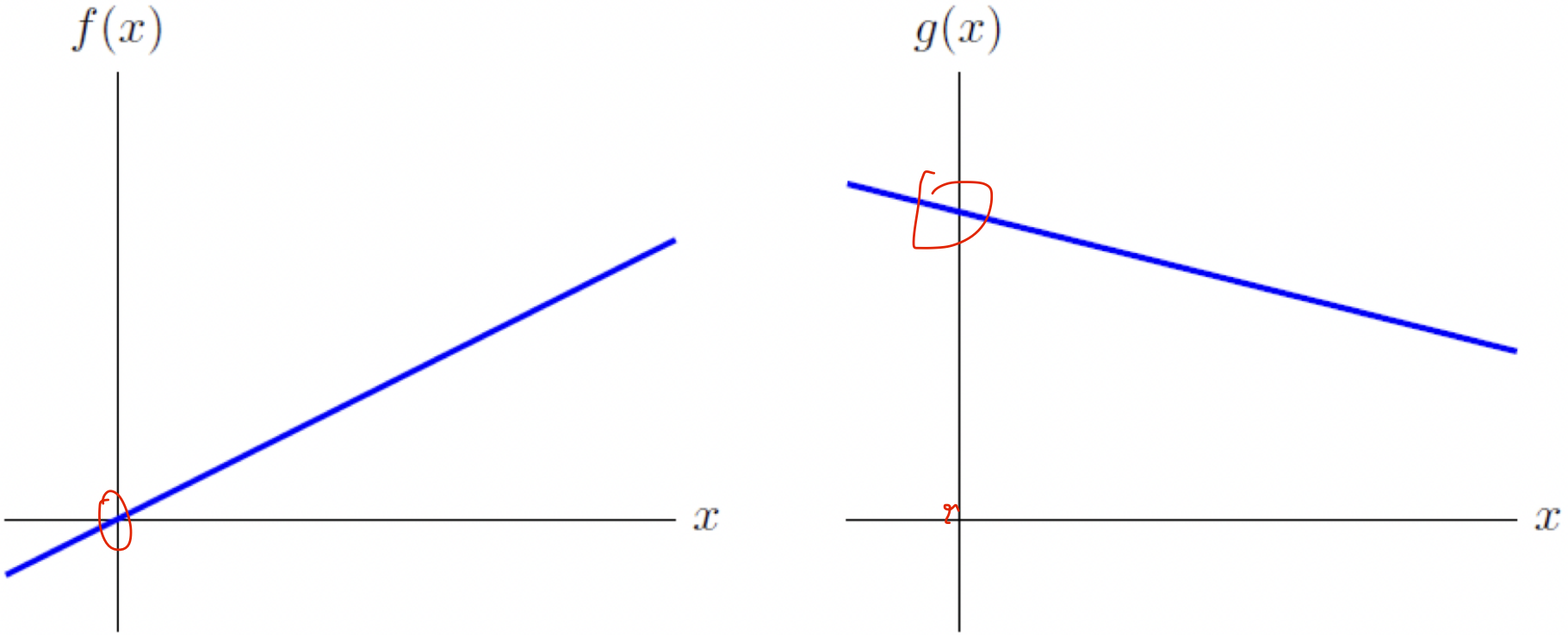
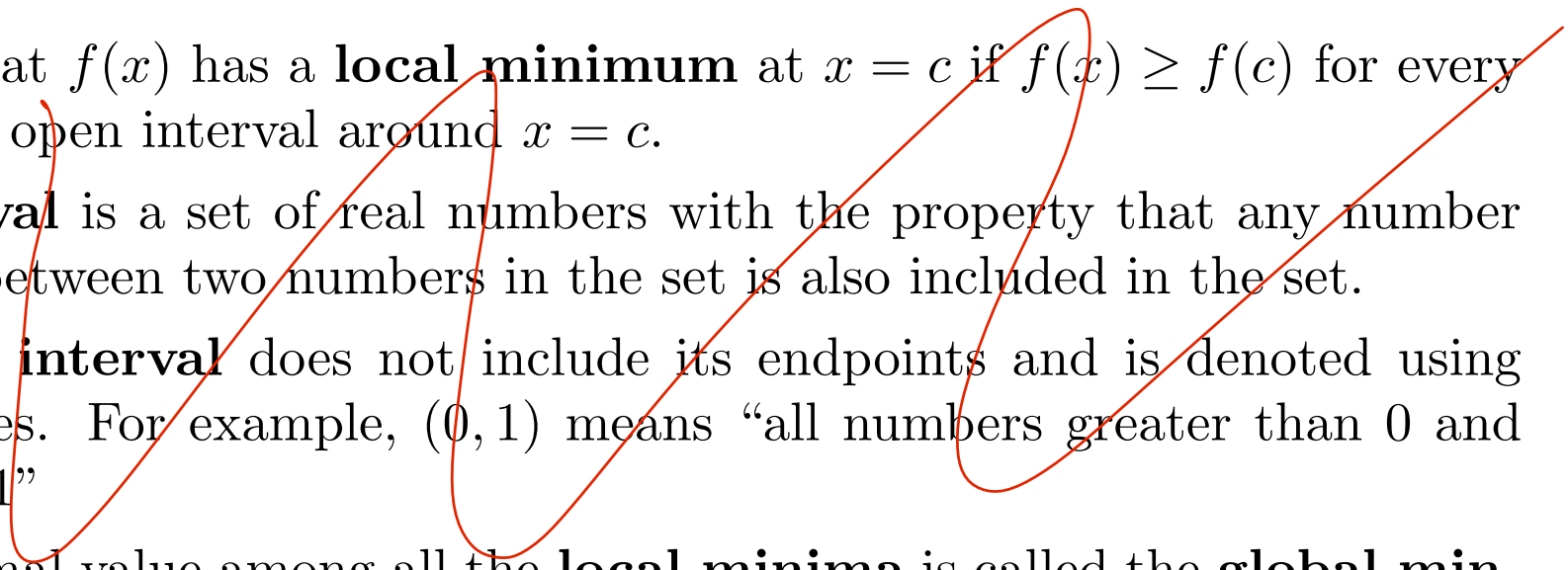


Figure 2.1 *Left.* The function f is linear. *Right.* The function g is affine, but not linear.

Functions, derivative and gradient

- We say that $f(x)$ has a **local minimum** at $x = c$ if $f(x) \geq f(c)$ for every x in some open interval around $x = c$.
- An **interval** is a set of real numbers with the property that any number that lies between two numbers in the set is also included in the set.
- An **open interval** does not include its endpoints and is denoted using parentheses. For example, $(0, 1)$ means "all numbers greater than 0 and less than 1"
- The minimal value among all the **local minima** is called the **global minima**. See the illustration in the Figure on the next page.



Functions, derivative and gradient

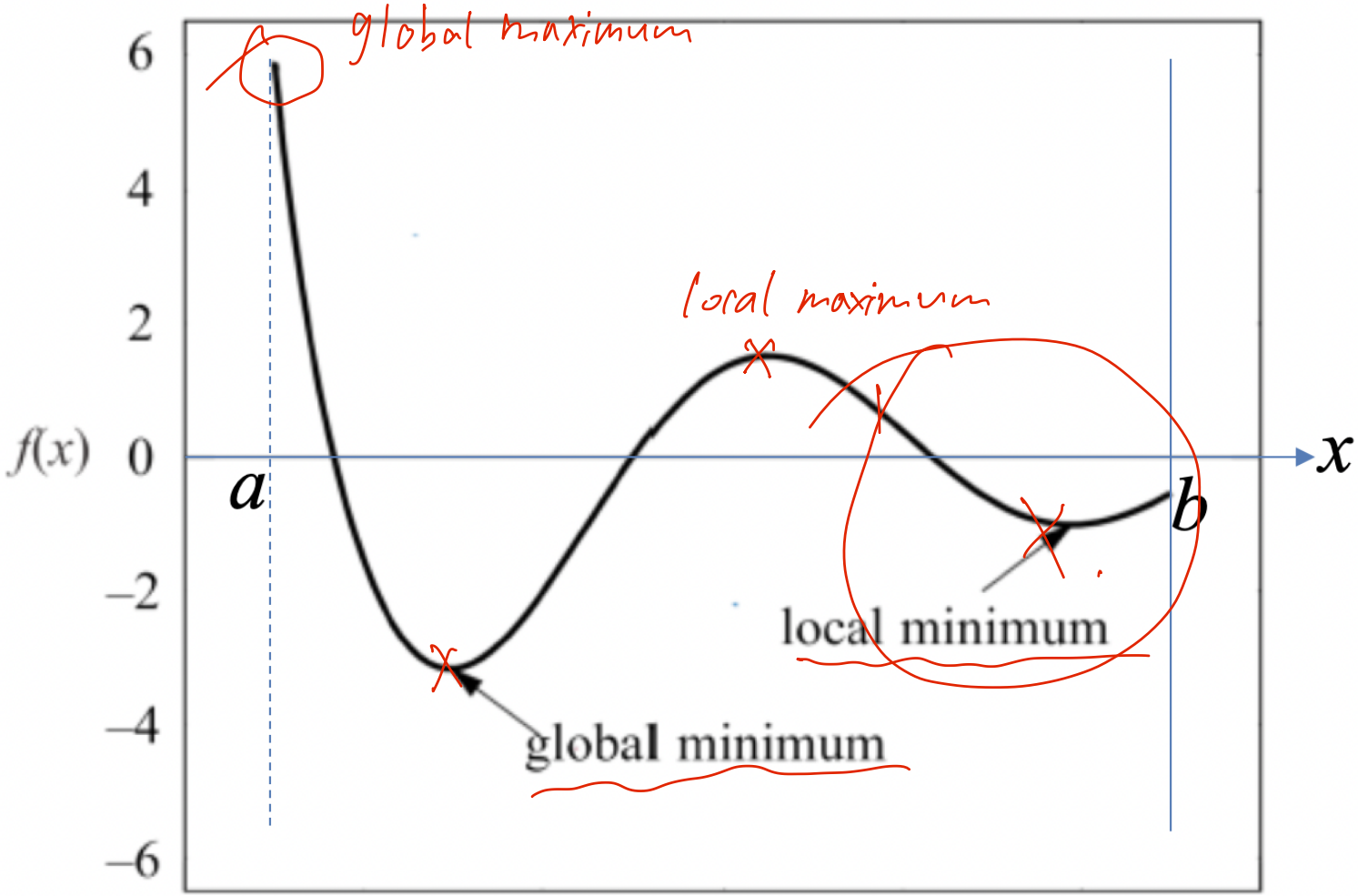


Figure: Local and global minima of a function. $a < x < b$

Functions, derivative and gradient

$0 = \max_x -(x-1)^2$

$1 = \arg\max_x -(x-1)^2$

max **vs** arg max

- Given a set of values $\mathcal{A} = \{a_1, a_2, ..., a_m\}$, the operator $\max_{a \in \mathcal{A}} f(a)$ returns the highest value $f(a)$ for all elements in the set $\mathcal{A}$.
- On the other hand, the operator $\arg\max_{a \in \mathcal{A}} f(a)$ returns the element of the set $\mathcal{A}$ that maximizes $f(a)$.
- Sometimes, ~~when the set is implicit or explicit~~, we can write

  there is no explicit constraint on a

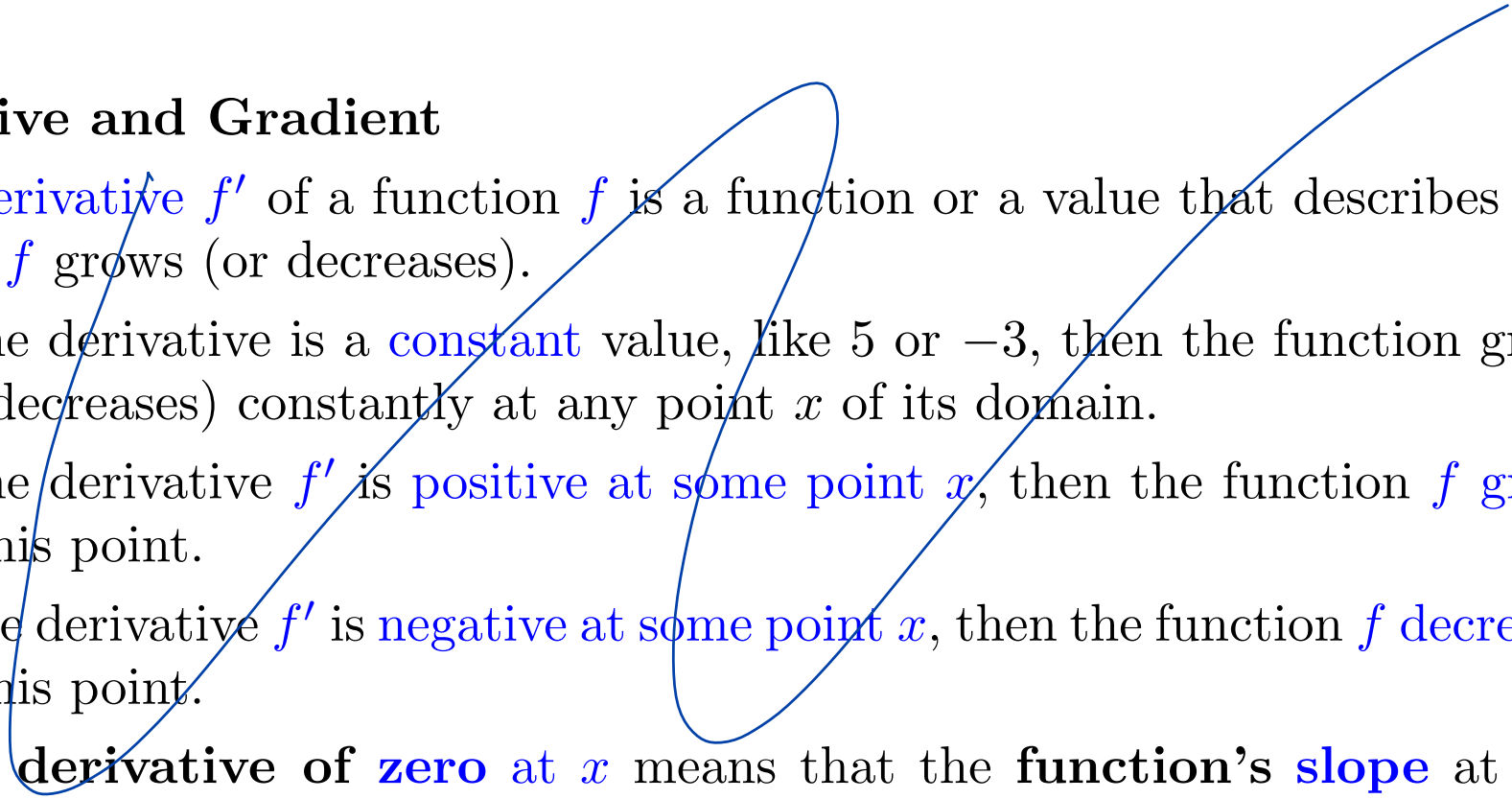
$$\max_a f(a) \quad \text{or} \quad \arg\max_a f(a)$$

- Operator min and arg min operates in a similar manner.
- Note: **arg max** returns a value from the **domain** of the function and **max** returns from the **range (codomain)** of the function

Functions, derivative and gradient

Derivative and Gradient

- A derivative f' of a function f is a function or a value that describes how fast f grows (or decreases).
- If the derivative is a constant value, like 5 or -3, then the function grows (or decreases) constantly at any point x of its domain.
- If the derivative f' is positive at some point x, then the function f grows at this point.
- If the derivative f' is negative at some point x, then the function f decreases at this point.
- The **derivative of zero** at x means that the **function's slope** at x is **horizontal**.



Functions, derivative and gradient

Partial Derivative

- **Differentiation of a scalar function *w.r.t.* a vector**
- If $f(\mathbf{w})$ is a scalar function of d variables, $\mathbf{w}$ is a $d \times 1$ vector, then differentiation of $f(\mathbf{w})$ *w.r.t.* $\mathbf{w}$ results in a $d \times 1$ vector.

$$\frac{df(\mathbf{w})}{d\mathbf{w}} = \begin{bmatrix} \frac{\partial f}{\partial w_1} \\ \vdots \\ \frac{\partial f}{\partial w_d} \end{bmatrix}$$

This is referred to as the **gradient** of $f(\mathbf{w})$ and written as $\nabla_{\mathbf{w}} f$.

Functions, derivative and gradient

Partial Derivative

- **Differentiation of a vector function *w.r.t.* a vector**
- If $\mathbf{f}(\mathbf{w})$ is a vector function of size $h \times 1$ and $\mathbf{w}$ is a $d \times 1$ vector, then differentiation of $\mathbf{f}(\mathbf{w})$ *w.r.t.* $\mathbf{w}$ results in a $d \times h$ vector.

$$\frac{d\mathbf{f}(\mathbf{w})}{d\mathbf{w}} = \begin{bmatrix} \frac{\partial f_1}{\partial w_1} & \cdots & \frac{\partial f_h}{\partial w_1} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_1}{\partial w_d} & \cdots & \frac{\partial f_h}{\partial w_d} \end{bmatrix}$$

- This is referred to as the **Jacobian** matrix of $\mathbf{f}(\mathbf{w})$, *i.e.*,

$$\mathbf{J} = \frac{d\mathbf{f}(\mathbf{w})}{d\mathbf{w}},$$

$$\mathbf{J}_{ij} = \frac{\partial f_j}{\partial w_i}.$$

Functions, derivative and gradient

Some Vector-Matrix Differentiation Formulations

$$\frac{d(\mathbf{X}^\top \mathbf{w})}{d\mathbf{w}} = \mathbf{X}, \text{where } \mathbf{X} \text{ is not a function of } \mathbf{w}$$

$$\frac{d(\mathbf{y}^\top \mathbf{X}\mathbf{w})}{d\mathbf{w}} = \mathbf{X}^\top \mathbf{y}$$

$$\frac{d(\mathbf{w}^\top \mathbf{X}\mathbf{w})}{d\mathbf{w}} = (\mathbf{X} + \mathbf{X}^\top)\mathbf{w}$$

- Note that we adopt the denominator layout derivative. If you use the numerator layout derivative, then all the above results will be transposed.
- Both types are OK, but keep it **consistent** in all derivatives.
- Please refer to the following wiki page for more details:
  `https://en.wikipedia.org/wiki/Matrix_calculus`

1 Notations, vectors, matrices

2 Functions, derivative and gradient

3 Modeling of linear regression
- Deterministic perspective
- Probabilistic perspective

4 Learning of linear regression
- Analytical solution
- Gradient descent algorithm

5 Linear regression of multiple outputs

6 Linear regression for classification

7 Variants of linear regression
- Ridge regression
- Polynomial regression
- Lasso regression
- Robust linear regression

Linear regression

Dataset: A collection of m labeled examples $\{(\mathbf{x}_i, y_i)\}_{i=1}^m$, with $\mathbf{x}_i \in \mathcal{X}$ being the d-dimensional feature vector of the i-th example, and $y_i \in \mathcal{Y}$ being a real-valued target.

Linear hypothesis function:

- We want to build a linear model, *i.e.*, a linear hypothesis function, as follows

$$f_{w_0,\ldots,w_d}(\mathbf{x}) = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_d x_d,$$

  where w_0 is often called bias and $w_1, \ldots, w_d$ are often called coefficients.
- Let $\mathbf{w} = [w_0, w_1, \ldots, w_d]^\top$ and rewrite $\mathbf{x} \leftarrow [1, x_1, \ldots, x_d]^\top$ (augmented feature vector). Then the linear model is equivalent to

$$f_{\mathbf{w}}(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}$$

- Note: $f_{\mathbf{w}}$ is called **linear** due to the linearity *w.r.t.* the parameter vector $\mathbf{w}$, rather than *w.r.t.* the original feature vector $[x_1, \ldots, x_d]^\top$.

Task of linear regression:

- Using $f_{\mathbf{w}}$ to approximate the ground-truth target function $t : \mathcal{X} \to \mathcal{Y}$.
- Note: If $\mathcal{Y}$ is a **finite and discrete** set, then the task corresponds to a classification problem; if $\mathcal{Y}$ is a **continuous** space, then the task corresponds to a regression problem.

Linear regression: deterministic perspective

$\min_{w} \frac{1}{m}\sum_{i=1}^{m} l(y_i, \hat{y}_i)$ $\quad \hat{y}_i = f_w(x_i)$

$l(y, \hat{y}) = (y-\hat{y})^2$ square loss

$l(y, \hat{y}) = |y-\hat{y}|$ abs loss

Learning objective function

- To find the optimal values for $\mathbf{w}$ which minimizes the following expression:

$$\frac{1}{m}\sum_{i=1}^{m}\left(f_{\mathbf{w}}\left(\mathbf{x}_i\right)-y_i\right)^2$$

MSE

- In mathematics, the expression we minimize or maximize is called an objective function, or, simply, an objective.

Linear regression: deterministic perspective

- The expression $\left(f_{\mathbf{w}}\left(\mathbf{x}_i\right) - y_i\right)^2$ in the above objective is called the loss function. It's a penalty measure of mismatch for example i.
- This particular choice of the loss function is called squared error loss.
- All model-based learning algorithms have a loss function and what we do to find the best model is we try to minimize the objective known as the cost function.
- In linear regression, the cost function is given by the average loss, also called the empirical risk.

$\mathbb{E}_{(x,y)}(y - f_w(x))^2$: true risk

Linear regression: probabilistic perspective

- We assume that the relationship between the input variable/feature $\mathbf{x}$ and the output variable y is

$$y = \mathbf{w}^\top \mathbf{x} + e, \text{ where } e \sim \mathcal{N}(0, \sigma^2), \tag{1}$$

  where e is called observation noise or residual error, and it is independent with any specific input $\mathbf{x}$.

- Thus, the output y can also be seen as a random variable, and its conditional probability is formulated as

$$p(y|\mathbf{x}, \mathbf{w}) = \mathcal{N}(\mathbf{w}^\top \mathbf{x}, \sigma^2) \tag{2}$$

Linear regression: probabilistic perspective

Maximum log-likelihood estimation:

- The parameter $\mathbf{w}$ can be learned by maximum log-likelihood estimation (MLE), given the training dataset $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^m$, as follows

$$\mathbf{w}_{MLE} = \arg\max_{\mathbf{w}} \log \mathcal{L}(\mathbf{w}; D), \tag{3}$$

$$\log \mathcal{L}(\mathbf{w}; D) = \log \left(\prod_{i=1}^m p(y_i|\mathbf{x}_i, \mathbf{w}) \right) = \sum_{i=1}^m \log \mathcal{N}(\mathbf{w}^\top \mathbf{x}_i, \sigma^2) \tag{4}$$

$$= -m \log(\sigma(2\pi)^{\frac{1}{2}}) - \frac{1}{2\sigma^2} \sum_{i=1}^m (y_i - \mathbf{w}^\top \mathbf{x}_i)^2.$$

- Removing the constants *w.r.t.* $\mathbf{w}$,

$$\mathbf{w}_{MLE} = \arg\min_{\mathbf{w}} \frac{1}{2} \sum_{i=1}^m (y_i - \mathbf{w}^\top \mathbf{x}_i)^2, \tag{5}$$

which is exactly the same with the cost function from the deterministic perspective.

1 Notations, vectors, matrices

2 Functions, derivative and gradient

3 Modeling of linear regression
- Deterministic perspective
- Probabilistic perspective

4 Learning of linear regression
- Analytical solution
- Gradient descent algorithm

5 Linear regression of multiple outputs

6 Linear regression for classification

7 Variants of linear regression
- Ridge regression
- Polynomial regression
- Lasso regression
- Robust linear regression

Linear regression with analytical solution

Learning (Training)

- Consider the set of feature vector $\mathbf{x}_i$ and target output y_i indexed by $i = 1, \ldots, m$, then a linear model $f_{\mathbf{w}}(\mathbf{x}) = \mathbf{x}^\top \mathbf{w} = \sum_{j=0}^{d} w_j x_j$ can be packed as

$$\hat{y} = f_{\mathbf{w}}(\mathbf{X}) \quad \Leftrightarrow \quad \mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_m \end{bmatrix}$$

Learning model — Learning target vector

$$= \begin{bmatrix} \mathbf{x}_1^\top \mathbf{w} \\ \vdots \\ \mathbf{x}_m^\top \mathbf{w} \end{bmatrix}$$

$$y_i = x_i^\top w$$

where $\mathbf{x}_i^\top \mathbf{w} = [1, x_1, \ldots, x_d]_i \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_d \end{bmatrix}$ and $\mathbf{X} = \begin{bmatrix} \mathbf{x}_1^\top \\ \vdots \\ \mathbf{x}_m^\top \end{bmatrix}$

Note: The **bias term** is responsible for **shifting the line/plane** up or down.

Linear Regression

Least Squares Regression

- In vector-matrix notation, the squared error function for all the m samples can be written compactly using $\mathbf{e} = \mathbf{X}\mathbf{w} - \mathbf{y}$:

$$
\begin{aligned}
J(\mathbf{w}) &= \sum_{i=1}^{m} \left(f_{\mathbf{w}}\left(\mathbf{x}_i\right) - y_i\right)^2 \\
&= \sum_{i=1}^{m} e_i^2 \\
&= \mathbf{e}^\top \mathbf{e} \\
&= (\mathbf{X}\mathbf{w} - \mathbf{y})^\top (\mathbf{X}\mathbf{w} - \mathbf{y}) \\
&= \left(\mathbf{w}^\top \mathbf{X}^\top - \mathbf{y}^\top\right)(\mathbf{X}\mathbf{w} - \mathbf{y}) \\
&= \mathbf{w}^\top \mathbf{X}^\top \mathbf{X}\mathbf{w} - \mathbf{w}^\top \mathbf{X}^\top \mathbf{y} - \mathbf{y}^\top \mathbf{X}\mathbf{w} + \mathbf{y}^\top \mathbf{y} \\
&= \mathbf{w}^\top \mathbf{X}^\top \mathbf{X}\mathbf{w} - 2\mathbf{y}^\top \mathbf{X}\mathbf{w} + \mathbf{y}^\top \mathbf{y}
\end{aligned}
$$

Linear Regression

Differentiating $J(\mathbf{w})$ with respect to $\mathbf{w}$ and setting the result to 0 :

$$\begin{aligned}
\frac{\partial}{\partial \mathbf{w}} J(\mathbf{w}) &= \mathbf{0} \\
\frac{\partial}{\partial \mathbf{w}} \left(\mathbf{w}^\top \mathbf{X}^\top \mathbf{X}\mathbf{w} - 2\mathbf{y}^\top \mathbf{X}\mathbf{w} + \mathbf{y}^\top \mathbf{y}\right) &= \mathbf{0} \\
\Rightarrow 2\mathbf{X}^\top \mathbf{X}\mathbf{w} - 2\mathbf{X}^\top \mathbf{y} &= \mathbf{0} \\
\Rightarrow 2\mathbf{X}^\top \mathbf{X}\mathbf{w} &= 2\mathbf{X}^\top \mathbf{y}
\end{aligned}$$

If $\mathbf{X}^\top \mathbf{X}$ is invertible, then

$$\begin{aligned}
&\text{Learning:} \quad \widehat{\mathbf{w}} = \left(\mathbf{X}^\top \mathbf{X}\right)^{-1} \mathbf{X}^\top \mathbf{y} \\
&\text{Prediction:} \quad f_{\mathbf{w}}\left(\mathbf{X}_{\text{new}}\right) = \mathbf{X}_{\text{new}}\, \widehat{\mathbf{w}}
\end{aligned}$$

Linear Regression

Example 1

$\{(\mathbf{x}_i, y_i)\}_{i=1}^{m}$

$$\{x = -9\} \rightarrow \{y = -6\}$$
$$\{x = -7\} \rightarrow \{y = -6\}$$
$$\{x = -5\} \rightarrow \{y = -4\}$$
$$\{x = 1\} \rightarrow \{y = -1\}$$
$$\{x = 5\} \rightarrow \{y = 1\}$$
$$\{x = 9\} \rightarrow \{y = 4\}$$

$$\begin{bmatrix} 1 & -9 \\ 1 & -7 \\ 1 & -5 \\ 1 & 1 \\ 1 & 5 \\ 1 & 9 \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \end{bmatrix} = \begin{bmatrix} -6 \\ -6 \\ -4 \\ -1 \\ 1 \\ 4 \end{bmatrix}$$

$\mathbf{X}$ $\quad \mathbf{w} = \mathbf{y}$

(Least squares approximation).

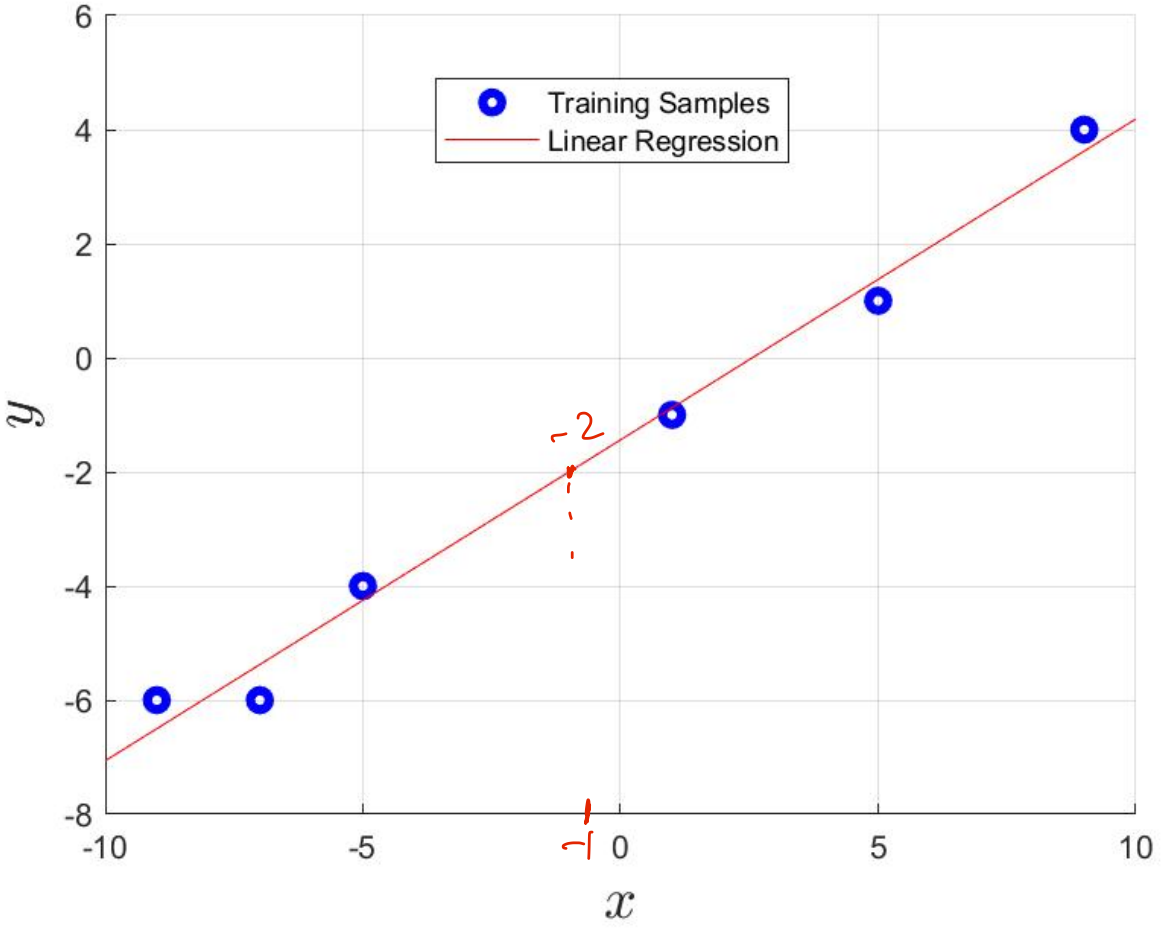
This set of linear equations has NO exact solution.

$$\hat{\mathbf{w}} = \mathbf{X}^{\dagger}\mathbf{y} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

However ($\mathbf{X}^T\mathbf{X}$ is invertible)

$$= \begin{bmatrix} 6 & -6 \\ -6 & 262 \end{bmatrix}^{-1} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 \\ -9 & -7 & -5 & 1 & 5 & 9 \end{bmatrix} \begin{bmatrix} -6 \\ -6 \\ -4 \\ -1 \\ 1 \\ 4 \end{bmatrix} = \begin{bmatrix} -1.4375 \\ 0.5625 \end{bmatrix}$$

Linear regression



$$\hat{y} = \mathbf{X}\hat{\mathbf{w}}$$

$$= \mathbf{X}\begin{bmatrix} -1.4375 \\ 0.5625 \end{bmatrix}$$

$$y = -1.4375 + 0.5625x$$

-1

Prediction:

$$\{x = -1\} \rightarrow \{y = ?\}$$

$$\hat{y} = [1 \; -1]\begin{bmatrix} -1.4375 \\ 0.5625 \end{bmatrix}$$

$$= -2$$

Linear Regression for one-dimensional examples.

Linear Regression

Example 2

$\{(\mathbf{x}_i, y_i)\}_{i=1}^m$

$\{x_1 = 1, x_2 = 1, x_3 = 1\} \rightarrow \{y = 1\}$
$\{x_1 = 1, x_2 = -1, x_3 = 1\} \rightarrow \{y = 0\}$
$\{x_1 = 1, x_2 = 1, x_3 = 3\} \rightarrow \{y = 2\}$
$\{x_1 = 1, x_2 = 1, x_3 = 0\} \rightarrow \{y = -1\}$

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & -1 & 1 \\ 1 & 1 & 3 \\ 1 & 1 & 0 \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 2 \\ -1 \end{bmatrix}$$

$\mathbf{X}$ $\mathbf{w}$ $\mathbf{y}$

$\mathbf{w}^T\mathbf{X}^T$

This set of linear equations has NO exact solution.

$$\hat{\mathbf{w}} = \mathbf{X}^{\dagger}\mathbf{y} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

However ($\mathbf{X}^T\mathbf{X}$ is invertible)

$$= \begin{bmatrix} 4 & 2 & 5 \\ 2 & 4 & 3 \\ 5 & 3 & 11 \end{bmatrix}^{-1} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & 1 \\ 1 & 1 & 3 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 2 \\ -1 \end{bmatrix} \approx \begin{bmatrix} -0.75 \\ 0.18 \\ 0.93 \end{bmatrix}$$ (Least squares approximation).

Linear Regression

Prediction:

$$\{x_1 = 1, x_2 = 6, x_3 = 8\} \rightarrow \{y = ?\}$$
$$\{x_1 = 1, x_2 = 0, x_3 = -1\} \rightarrow \{y = ?\}$$

$$\hat{y} = \begin{bmatrix} 1 & 6 & 8 \\ 1 & 0 & -1 \end{bmatrix} \begin{bmatrix} -0.75 \\ 0.18 \\ 0.93 \end{bmatrix}$$

$$= \begin{bmatrix} 7.7500 \\ -1.6786 \end{bmatrix}$$

Linear regression solved by gradient descent

- The linear regression is formulated for the following optimization problem

$$\mathbf{w}^* = \arg\min_{\mathbf{w}} J(\mathbf{w}), \quad J(\mathbf{w}) = \frac{1}{2}\sum_{i=1}^{m}(\mathbf{x}_i^\top \mathbf{w} - y_i)^2 = \frac{1}{2}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 \tag{6}$$

- $\mathbf{w}$ can be updated by gradient descent algorithm,

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}}, \quad \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}} = \mathbf{X}^\top(\mathbf{X}\mathbf{w} - \mathbf{y}) \tag{7}$$

  where α is called step size or learning rate.

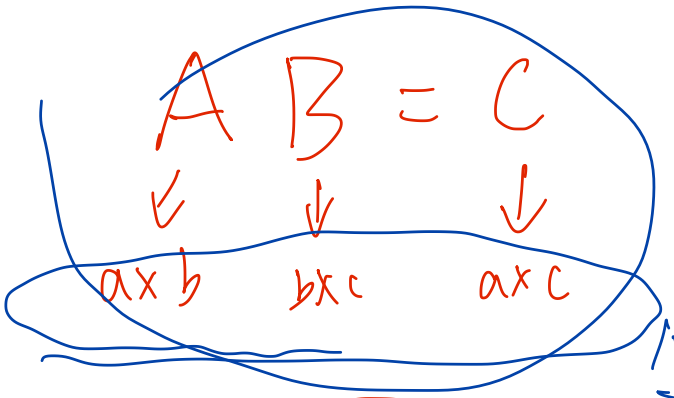
- Does gradient descent always converge to the optimal solution? (Plot the update trajectory of gradient descent on loss curve)

Closed-form solution vs. gradient descent

Closed-form solution $\mathbf{w} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$	**Gradient descent** $\mathbf{w} \leftarrow \mathbf{w} - \alpha \mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y})$
No hyper-parameter	Needs to choose α
No need to iterate	Needs many iterations
Complexity $O(d^3 + md^2)$	Complexity $O(T \times md^2)$
Slow if d is very large	Works well when d is large

Thus, you can choose between the above two solutions according to the dimensionality of your training data:

- When the training data is very high-dimensional, *i.e.*, d is very **large**, then it is better to choose **gradient descent algorithm**
- When the training data is not high-dimensional, *i.e.*, d is very **small**, then it is better to choose **closed-form solution**



time complexity

$O(abc)$

not $O(ab^2c)$

C | C_{ij}

$C_{ij} = a_{i:} \cdot b_{:j}$

$= \sum_{k=1}^{b} a_{ik} b_{kj} \rightarrow O(1)$

$y = 0.7ab^2$

$y = 0(ab^2)$

$y = 100\,ab^2$

$y = 0(ab^2)$

$\downarrow O(b) + O(b)$

$\downarrow O(b)$

space complexity

$X : m \times n$

$O(mn)$

$w \in w - \alpha X^T (Xw - y)$ (mx1)

$d \times m \quad O(md) + O(m)$

$O(md)$

T iterations $O(mdT)$

$w \in w - \alpha (X^T X w - X^T y)$

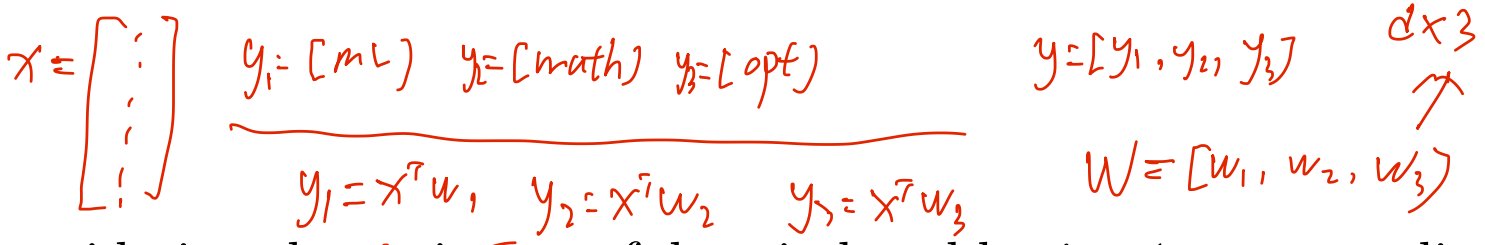
$\downarrow$

$O(md^2) + O(d^2) + O(dm)$

T: $O(md^2T)$

1. Notations, vectors, matrices
2. Functions, derivative and gradient
3. Modeling of linear regression
   - Deterministic perspective
   - Probabilistic perspective
4. Learning of linear regression
   - Analytical solution
   - Gradient descent algorithm
5. Linear regression of multiple outputs
6. Linear regression for classification
7. Variants of linear regression
   - Ridge regression
   - Polynomial regression
   - Lasso regression
   - Robust linear regression

Linear regression with single output



When considering the entire set of data indexed by $i = 1, \ldots, m$, a linear model $f_{\mathbf{w}}(\mathbf{x}) = \mathbf{x}^\top \mathbf{w} = \sum_{j=0}^{d} w_j x_j$, where $x_0 \equiv 1$, can be packed as $y = x^T W$

$$f_{\mathbf{w}}(\mathbf{X}) = \mathbf{X}\mathbf{w} \leftarrow \text{Scalar function}$$

$$= \begin{bmatrix} \mathbf{x}_1^\top \mathbf{w} \\ \vdots \\ \mathbf{x}_m^\top \mathbf{w} \end{bmatrix} \quad \text{where} \quad \mathbf{x}_i^\top \mathbf{w} = [1, x_{i,1}, \ldots, x_{i,d}] \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_d \end{bmatrix} \tag{8}$$

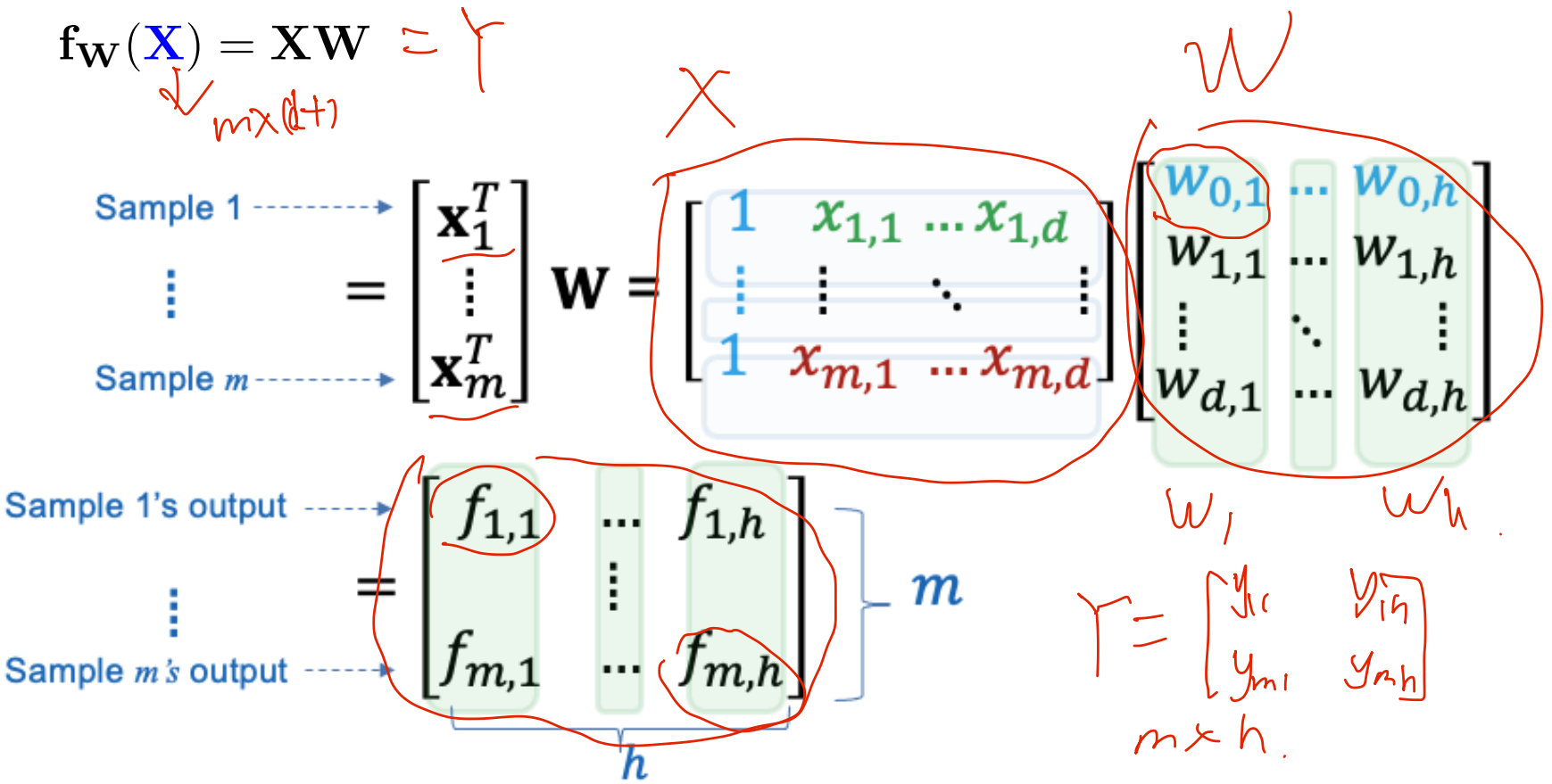
Primal solution: $\widehat{\mathbf{w}} = \left(\mathbf{X}^\top \mathbf{X}\right)^{-1} \mathbf{X}^\top \mathbf{y}$

Note: The **bias term** is responsible for **shifting the line/plane** up or down.

Linear regression with multiple outputs

- When considering the entire set of data indexed by $i = 1, \ldots, m$, a linear model $\mathbf{f}_{\mathbf{W}}(\mathbf{x}) = \mathbf{x}^\top \mathbf{W}$ can be packed as

$$\mathbf{f}_{\mathbf{W}}(\mathbf{X}) = \mathbf{X}\mathbf{W}$$



Linear regression with multiple outputs

In matrix-matrix notation, the squared error loss function can be written compactly using $\mathbf{E} = \mathbf{XW} - \mathbf{Y}$:

$$\begin{aligned} J(\mathbf{W}) &= \operatorname{trace}\left(\mathbf{E}^\top \mathbf{E}\right) \\ &= \operatorname{trace}\left[(\mathbf{XW} - \mathbf{Y})^\top (\mathbf{XW} - \mathbf{Y})\right] \end{aligned}$$

If $\mathbf{X}^\top \mathbf{X}$ is invertible, then

$$\text{Learning: } \widehat{\mathbf{W}} = \left(\mathbf{X}^\top \mathbf{X}\right)^{-1} \mathbf{X}^\top \mathbf{Y}$$

$$\text{Prediction: } \mathbf{f}_{\mathbf{W}}\left(\mathbf{X}_{\text{new}}\right) = \mathbf{X}_{\text{new}} \widehat{\mathbf{W}}$$

Assumption: the error terms $\mathbf{e}_k^\top \mathbf{e}_k$ are independent for all $k = 1, \ldots, h$

Linear regression with multiple outputs

$E = [e_1, e_2, \cdots, e_h]$

$$J(\mathbf{W}) = \text{trace}(\mathbf{E}^T\mathbf{E})$$

$$= \text{trace}\left(\begin{bmatrix}\mathbf{e}_1^T \\ \vdots \\ \mathbf{e}_h^T\end{bmatrix}[\mathbf{e}_1 \quad \mathbf{e}_2 \quad \dots \quad \mathbf{e}_h]\right)$$

$$= \text{trace}\left(\begin{bmatrix}\mathbf{e}_1^T\mathbf{e}_1 & \mathbf{e}_1^T\mathbf{e}_2 & \dots & \mathbf{e}_1^T\mathbf{e}_h \\ \mathbf{e}_2^T\mathbf{e}_1 & \mathbf{e}_2^T\mathbf{e}_2 & \dots & \mathbf{e}_2^T\mathbf{e}_h \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{e}_h^T\mathbf{e}_1 & \mathbf{e}_h^T\mathbf{e}_2 & \dots & \mathbf{e}_h^T\mathbf{e}_h\end{bmatrix}\right) = \sum_{k=1}^{h} \mathbf{e}_k^T\mathbf{e}_k$$

$\|E\|_F^2 = \sum_{j=1}^{h} \|e_j\|^2$

Linear regression

Example

$$\{\mathbf{x}_i, \mathbf{y}_i\}_{i=1}^m$$

$$\{x_1 = 1, x_2 = 1, x_3 = 1\} \to \{y_1 = 1, \quad y_2 = 0\}$$
$$\{x_1 = 1, x_2 = -1, x_3 = 1\} \to \{y_1 = 0, \quad y_2 = 1\}$$
$$\{x_1 = 1, x_2 = 1, \quad x_3 = 3\} \to \{y_1 = 2, y_2 = -1\}$$
$$\{x_1 = 1, x_2 = 1, \quad x_3 = 0\} \to \{y_1 = -1, y_2 = 3\}$$

$$\underbrace{\begin{bmatrix} 1 & 1 & 1 \\ 1 & -1 & 1 \\ 1 & 1 & 3 \\ 1 & 1 & 0 \end{bmatrix}}_{\mathbf{X}} \underbrace{\begin{bmatrix} w_{1,1} & w_{1,2} \\ w_{2,1} & w_{2,2} \\ w_{3,1} & w_{3,2} \end{bmatrix}}_{\mathbf{W}} = \underbrace{\begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 2 & -1 \\ -1 & 3 \end{bmatrix}}_{\mathbf{Y}}$$

This set of linear equations has NO exact solution.

$$\hat{\mathbf{W}} = \mathbf{X}^{\dagger}\mathbf{Y} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{Y}$$

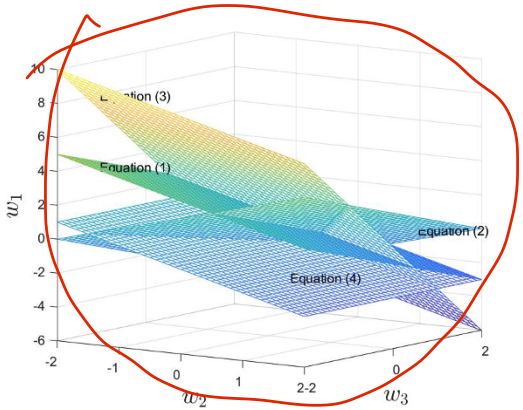
However ($\mathbf{X}^T\mathbf{X}$ is invertible)

$$= \begin{bmatrix} 4 & 2 & 5 \\ 2 & 4 & 3 \\ 5 & 3 & 11 \end{bmatrix}^{-1} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & 1 \\ 1 & 1 & 3 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 2 & -1 \\ -1 & 3 \end{bmatrix} = \begin{bmatrix} -0.75 & 2.25 \\ 0.1786 & 0.0357 \\ 0.9286 & -1.2143 \end{bmatrix}$$

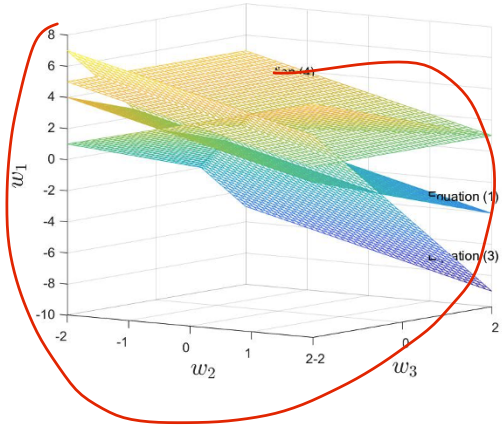
(Least squares approximation).

Linear regression with multiple outputs

Example:



y_1



y_2

Prediction:

$$\{x_1 = 1, x_2 = 6, \quad x_3 = 8\} \rightarrow \{y_1 = ?, y_2 = ?\}$$

$$\{x_1 = 1, x_2 = 0, x_3 = -1\} \rightarrow \{y_1 = ?, y_2 = ?\}$$

$$\hat{\mathbf{Y}} = \mathbf{X}_t \mathbf{W}$$

$$= \begin{bmatrix} 1 & 6 & 8 \\ 1 & 0 & -1 \end{bmatrix} \begin{bmatrix} -0.75 & 2.25 \\ 0.1786 & 0.0357 \\ 0.9286 & -1.2143 \end{bmatrix}$$

$$= \begin{bmatrix} 7.75 & -7.25 \\ -1.6786 & 3.4643 \end{bmatrix}$$

1. Notations, vectors, matrices
2. Functions, derivative and gradient
3. Modeling of linear regression
   - Deterministic perspective
   - Probabilistic perspective
4. Learning of linear regression
   - Analytical solution
   - Gradient descent algorithm
5. Linear regression of multiple outputs
6. Linear regression for classification
7. Variants of linear regression
   - Ridge regression
   - Polynomial regression
   - Lasso regression
   - Robust linear regression

Linear regression for classification

Dataset:

- We have a collection of m labeled examples $\{(\mathbf{x}_i, y_i)\}_{i=1}^m$, with $\mathbf{x}_i \in \mathcal{X}$ being the d-dimensional feature vector of the i-th example, and $y_i \in \mathcal{Y}$ being a real-valued target.

Linear hypothesis function:

- We want to build a linear model $f_{\mathbf{w}}(\mathbf{x})$, *i.e.*, linear hypothesis function,

$$f_{\mathbf{w}}(\mathbf{x}) = \mathbf{x}^\top \mathbf{w},$$

where $\mathbf{w} = [w_0, w_1, \ldots, w_d]^\top$ is a $(d+1)$-dimensional vector of parameters, w_0 is the bias parameter, and $\mathbf{x}$ is the $(d+1)$-dimensional augmented feature vector, i.e., $\mathbf{x} = [1, x_1, \ldots, x_d]^\top$.

Task of linear regression:

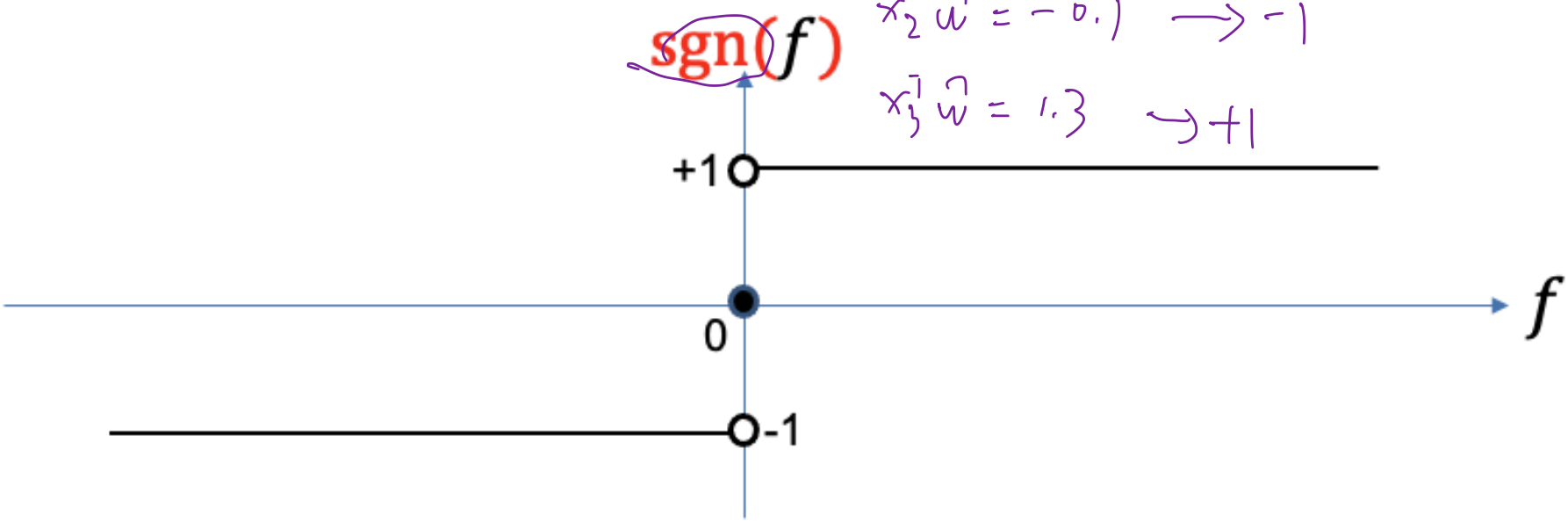
- Using the linear model $f_{\mathbf{w}}$ to approximate the ground-truth target function $t : \mathcal{X} \to \mathcal{Y}$.
- Note: If $\mathcal{Y}$ is a **finite and discrete** set, then the task corresponds to a **classification** problem; if $\mathcal{Y}$ is a **continuous** space, then the task corresponds to a regression problem.

Linear regression for classification

Binary Classification: If $\mathbf{X}^\top \mathbf{X}$ is invertible, then

Learning: $\widehat{\mathbf{w}} = \left(\mathbf{X}^\top \mathbf{X}\right)^{-1} \mathbf{X}^\top \mathbf{y}$, $y_i \in \{-1, +1\}, i = 1, \ldots, m$

Prediction: $f_{\mathbf{w}}(\mathbf{x}_{new}) = \operatorname{sgn}\left(\mathbf{x}_{new}^\top \widehat{\mathbf{w}}\right)$ (for each row $\mathbf{x}_{new}^\top$ of $\mathbf{X}_{new}$)



Linear regression for classification

Example

$\{\mathbf{x}_i, y_i\}_{i=1}^m$

$\{x=-9\} \to \{y=-1\}$
$\{x=-7\} \to \{y=-1\}$
$\{x=-5\} \to \{y=-1\}$
$\{x=1\} \to \{y=+1\}$
$\{x=5\} \to \{y=+1\}$
$\{x=9\} \to \{y=+1\}$

$$\begin{bmatrix} 1 & -9 \\ 1 & -7 \\ 1 & -5 \\ 1 & 1 \\ 1 & 5 \\ 1 & 9 \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \end{bmatrix} = \begin{bmatrix} -1 \\ -1 \\ -1 \\ 1 \\ 1 \\ 1 \end{bmatrix}$$

$\mathbf{X}$ $\mathbf{w}$ $\mathbf{y}$

This set of linear equations has NO exact solution.

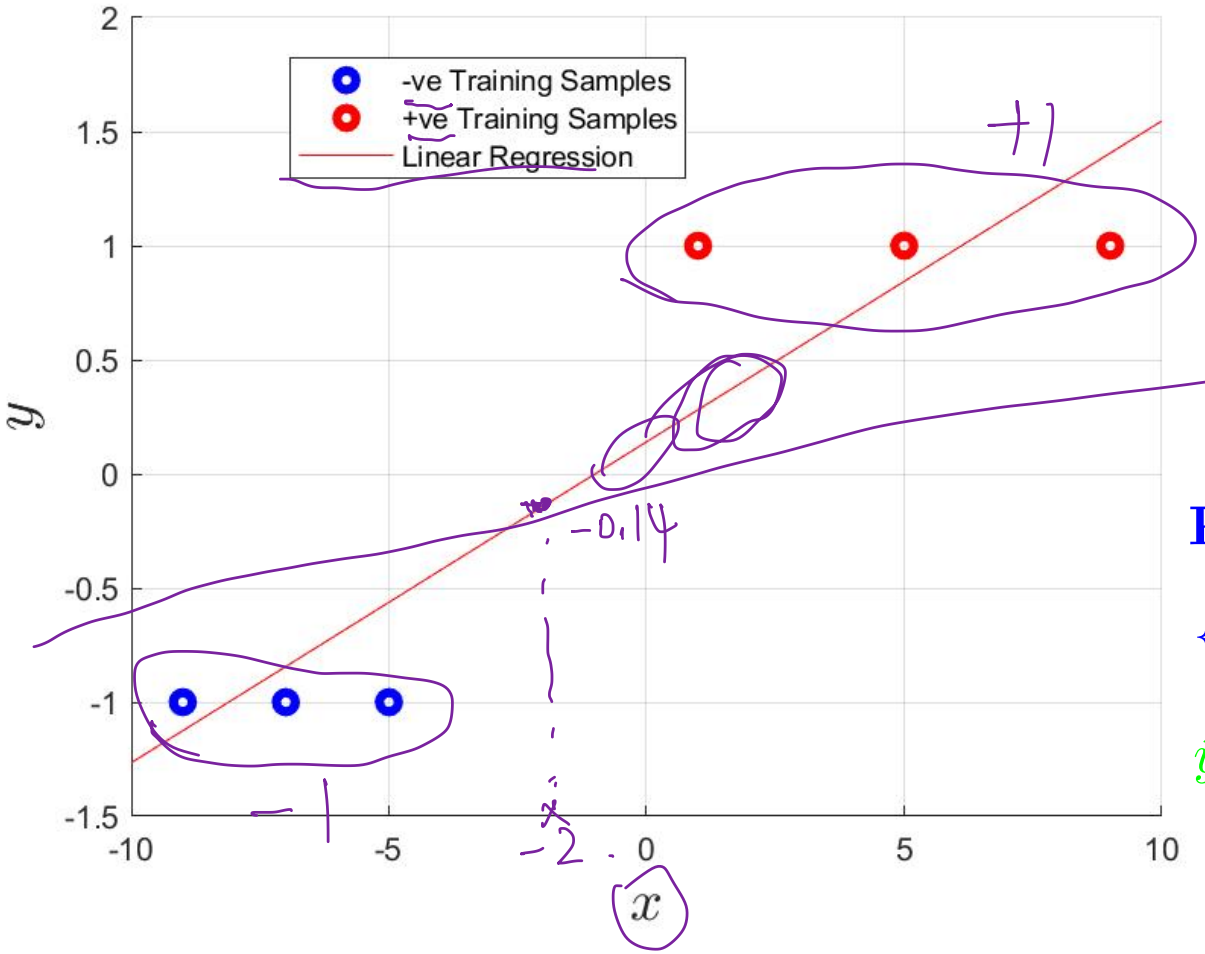
However ($\mathbf{X}^T\mathbf{X}$ is invertible)

$$\hat{\mathbf{w}} = \mathbf{X}^\dagger \mathbf{y} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

$$= \begin{bmatrix} 6 & -6 \\ -6 & 262 \end{bmatrix}^{-1} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 \\ -9 & -7 & -5 & 1 & 5 & 9 \end{bmatrix} \begin{bmatrix} -1 \\ -1 \\ -1 \\ 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.1406 \\ 0.1406 \end{bmatrix}$$

(Least squares approximation).

Linear regression for classification



$$\hat{y} = \text{sgn}(\mathbf{X}\widehat{\mathbf{w}})$$

$$= \text{sgn}\left(\mathbf{X}\begin{bmatrix} 0.1406 \\ 0.1406 \end{bmatrix}\right)$$

$$y = 0.1406 + 0.1406x$$

Prediction:

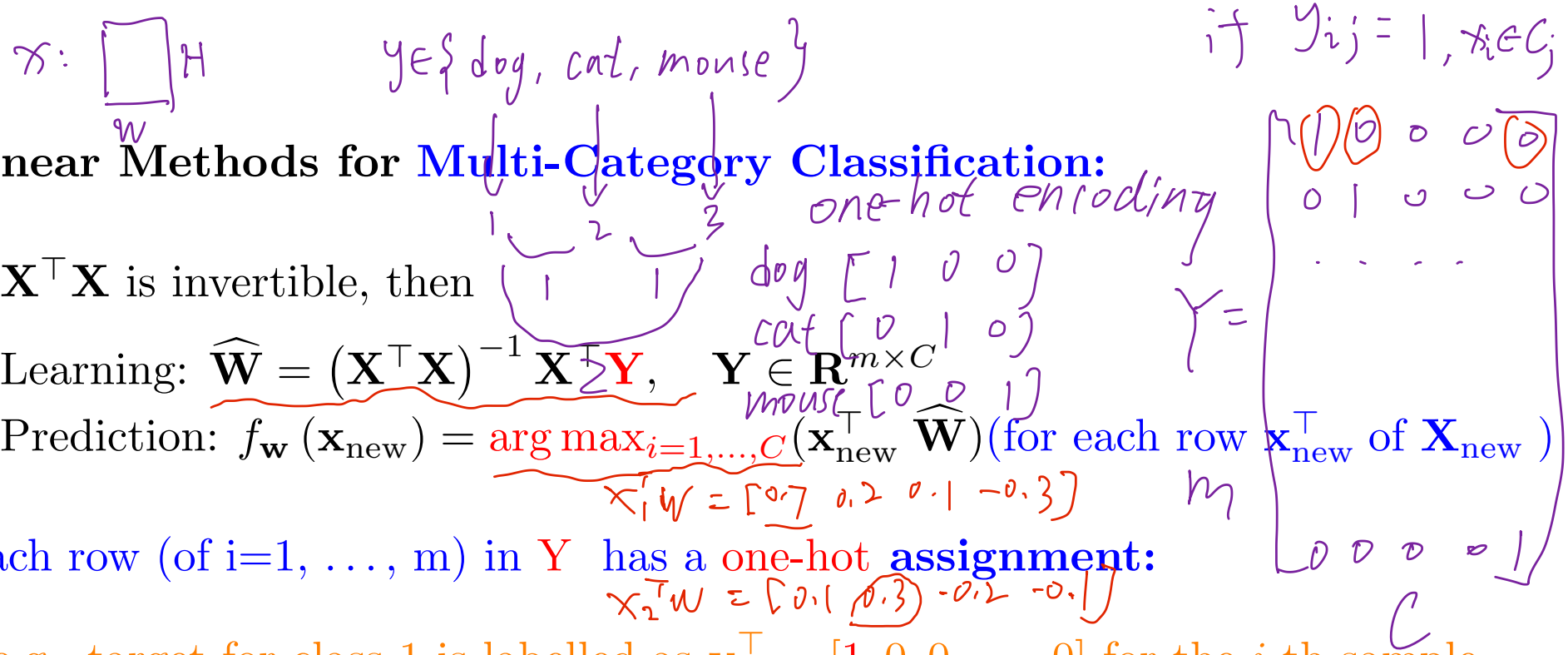
$$\{x = -2\} \rightarrow \{y = ?\}$$

$$\hat{y} = \text{sgn}\left([1 \; -2]\begin{bmatrix} 0.1406 \\ 0.1406 \end{bmatrix}\right)$$

$$= \text{sgn}(-0.1406) = -1$$

Linear regression for one-dimensional classification.

Linear regression for classification



Linear Methods for Multi-Category Classification:

If $\mathbf{X}^\top \mathbf{X}$ is invertible, then

Learning: $\widehat{\mathbf{W}} = \left(\mathbf{X}^\top \mathbf{X}\right)^{-1} \mathbf{X}^\top \mathbf{Y}, \quad \mathbf{Y} \in \mathbf{R}^{m \times C}$

Prediction: $f_{\mathbf{w}}\left(\mathbf{x}_{\text{new}}\right) = \arg\max_{i=1,\ldots,C} \left(\mathbf{x}_{\text{new}}^\top \widehat{\mathbf{W}}\right)$ (for each row $\mathbf{x}_{\text{new}}^\top$ of $\mathbf{X}_{\text{new}}$)

Each row (of i=1, . . . , m) in Y has a one-hot **assignment:**

e.g., target for class-1 is labelled as $\mathbf{y}_i^\top = [1, 0, 0, \ldots, 0]$ for the i th sample,

target for class-2 is labelled as $\mathbf{y}_i^\top = [0, 1, 0, \ldots, 0]$ for the i th sample,

target for class-C is labelled as $\mathbf{y}_i^\top = [0, 0, \ldots, 0, 1]$ for the i th sample.

Linear regression for classification

Example $\{\mathbf{x}_i, \mathbf{y}_i\}_{i=1}^m$

$$\{x_1 = 1,\ x_2 = 1,\ \ x_3 = 1\} \rightarrow \{y_1 = 1,\ y_2 = 0,\ y_3 = 0\}$$
$$\{x_1 = 1,\ x_2 = -1,\ x_3 = 1\} \rightarrow \{y_1 = 0,\ y_2 = 1,\ y_3 = 0\}$$
$$\{x_1 = 1,\ x_2 = 1,\ \ x_3 = 3\} \rightarrow \{y_1 = 1,\ y_2 = 0,\ y_3 = 0\}$$
$$\{x_1 = 1,\ x_2 = 1,\ \ x_3 = 0\} \rightarrow \{y_1 = 0,\ y_2 = 0,\ y_3 = 1\}$$

$$\underbrace{\begin{bmatrix} 1 & 1 & 1 \\ 1 & -1 & 1 \\ 1 & 1 & 3 \\ 1 & 1 & 0 \end{bmatrix}}_{\mathbf{X}} \underbrace{\begin{bmatrix} w_{1,1} & w_{1,2} & w_{1,3} \\ w_{2,1} & w_{2,2} & w_{2,3} \\ w_{3,1} & w_{3,2} & w_{3,3} \end{bmatrix}}_{\mathbf{W}} = \underbrace{\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}}_{\mathbf{Y}}$$

This set of linear equations has NO exact solution.

However ($\mathbf{X}^T\mathbf{X}$ is invertible) (Least squares approximation)

$$\widehat{\mathbf{W}} = \mathbf{X}^{\dagger}\mathbf{Y} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{Y}$$

$$= \begin{bmatrix} 4 & 2 & 5 \\ 2 & 4 & 3 \\ 5 & 3 & 11 \end{bmatrix}^{-1} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & 1 \\ 1 & 1 & 3 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0.5 & 0.5 \\ 0.2857 & -0.5 & 0.2143 \\ 0.2857 & 0 & -0.2857 \end{bmatrix}$$

Linear regression for classification

Example

Prediction:

$$\{x_1 = 1, x_2 = 6, \quad x_3 = 8\} \to \{ \text{ class } 1, 2, \text{ or } 3 \ ? \ \}$$
$$\{x_1 = 1, x_2 = 0, x_3 = -1\} \to \{ \text{ class } 1, 2, \text{ or } 3 \ ? \ \}$$

$$\widehat{\mathbf{Y}} = \mathbf{X}_t \widehat{\mathbf{W}} = \arg\max_{i=1,\ldots,C} \left(\begin{bmatrix} 1 & 6 & 8 \\ 1 & 0 & -1 \end{bmatrix} \begin{bmatrix} 0 & 0.5 & 0.5 \\ 0.2857 & -0.5 & 0.2143 \\ 0.2857 & 0 & -0.2857 \end{bmatrix} \right)$$

$$= \arg\max_{i=1,\ldots,C} \left(\begin{bmatrix} 4 & -2.50 & -0.50 \\ -0.2587 & 0.50 & 0.7857 \end{bmatrix} \right)$$

Position of the largest number determines the class label

$$= \begin{bmatrix} 1 \\ 3 \end{bmatrix} \begin{matrix} \to \text{Class-1} \\ \\ \to \text{ Class-3} \end{matrix}$$

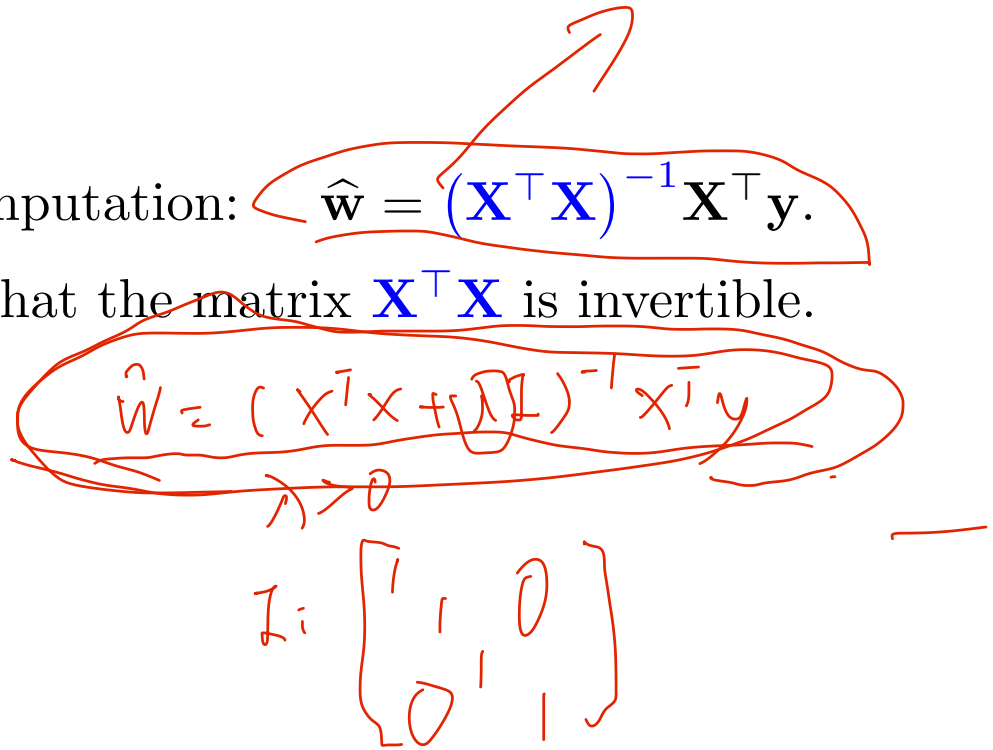
1. Notations, vectors, matrices
2. Functions, derivative and gradient
3. Modeling of linear regression
   - Deterministic perspective
   - Probabilistic perspective
4. Learning of linear regression
   - Analytical solution
   - Gradient descent algorithm
5. Linear regression of multiple outputs
6. Linear regression for classification
7. Variants of linear regression
   - Ridge regression
   - Polynomial regression
   - Lasso regression
   - Robust linear regression

Ridge regression

Motivation 1:

Recall the learning computation: $\widehat{\mathbf{w}} = \left(\mathbf{X}^\top \mathbf{X}\right)^{-1} \mathbf{X}^\top \mathbf{y}$.

We cannot guarantee that the matrix $\mathbf{X}^\top \mathbf{X}$ is invertible.



Ridge regression

Here on, we shall focus on single output y in derivations in the sequel.

$$\min_{\mathbf{w}} \sum_{i=1}^{m} \left(f_{\mathbf{w}}\left(\mathbf{x}_i\right) - y_i\right)^2 + \lambda \tilde{\mathbf{w}}^\top \tilde{\mathbf{w}}, \text{ where } \tilde{\mathbf{w}} = \hat{\mathbf{I}}_d \mathbf{w} = [0, w_1, w_2, \ldots, w_d]^\top,$$

$\hat{\mathbf{I}}_d \in \mathbb{R}^{(d+1)\times(d+1)}$ is defined by setting the $(1,1)$ entry in the $d+1$ dimensional identity matrix $\hat{\mathbf{I}}_{d+1}$ as 0. Note: The bias/offset w_0 is NOT included in the ℓ_2 regularization term, as it just affects the function's height, not its complexity.

Linear Model: $\min_{\mathbf{w}} (\mathbf{X}\mathbf{w} - \mathbf{y})^\top (\mathbf{X}\mathbf{w} - \mathbf{y}) + \lambda \tilde{\mathbf{w}}^\top \tilde{\mathbf{w}}$

$$\frac{\partial}{\partial \mathbf{w}} (\mathbf{X}\mathbf{w} - \mathbf{y})^\top (\mathbf{X}\mathbf{w} - \mathbf{y}) + \lambda \tilde{\mathbf{w}}^\top \tilde{\mathbf{w}} = \mathbf{0}$$

$$\Rightarrow 2\mathbf{X}^\top \mathbf{X}\mathbf{w} - 2\mathbf{X}^\top \mathbf{y} + 2\lambda \hat{\mathbf{I}}_d \mathbf{w} = \mathbf{0}$$

$$\Rightarrow \mathbf{X}^\top \mathbf{X}\mathbf{w} + \lambda \hat{\mathbf{I}}_d \mathbf{w} = \mathbf{X}^\top \mathbf{y}$$

$$\Rightarrow \left(\mathbf{X}^\top \mathbf{X} + \lambda \hat{\mathbf{I}}_d\right) \mathbf{w} = \mathbf{X}^\top \mathbf{y}$$

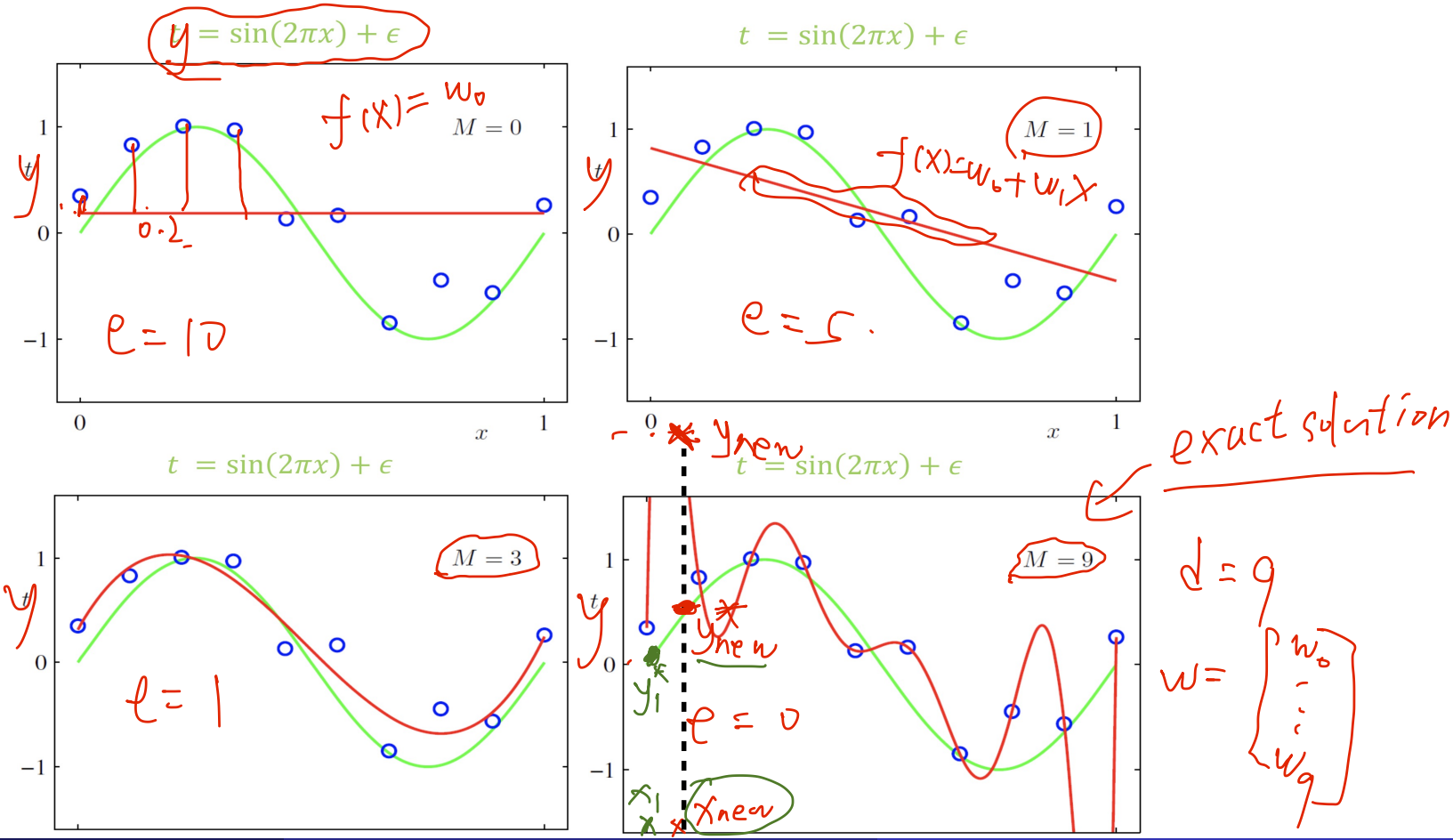
$$\Rightarrow \mathbf{w} = \left(\mathbf{X}^\top \mathbf{X} + \lambda \hat{\mathbf{I}}_d\right)^{-1} \mathbf{X}^\top \mathbf{y}$$

Note that $\left(\mathbf{X}^\top \mathbf{X} + \lambda \hat{\mathbf{I}}_d\right)$ is guaranteed to be invertible, given $\lambda > 0$.

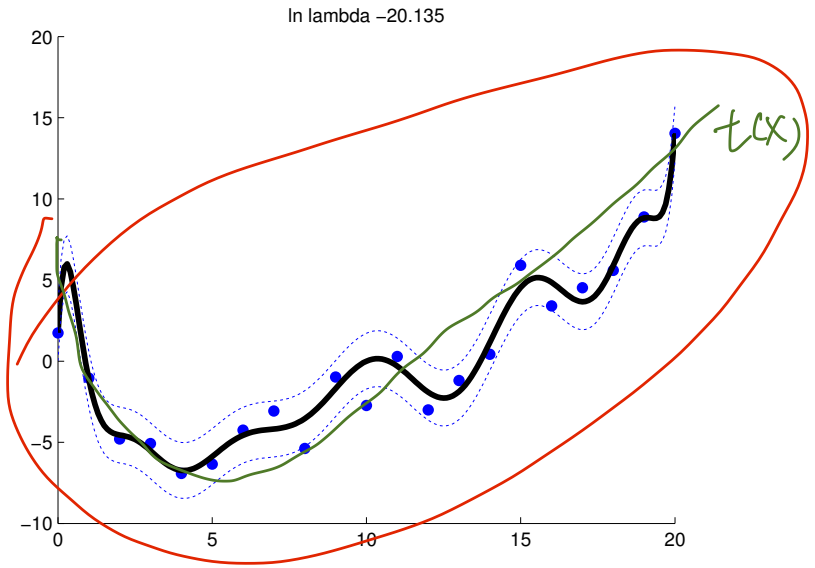
Ridge regression

Motivation 2: $f(x) = w_0 + w_1 x^1 + w_2 x^2 + w_3 x^3 + w_4 x^4 + \cdots + w_M x^M$

- Overfitting is an important challenge for linear regression, as shown below. Note: M in the figure denotes the degree of polynomial hypothesis function.
- If overfitting, the prediction performance on testing data will be very poor. How to alleviate overfitting?



Ridge regression



- Let's see one simple example, we use a **polynomial function** (introduced later) with 14 degrees to fit $m = 21$ data points. The learned curve is very "wiggly" (see above).
- The parameter values of this curve are as follows

$$6.56, -36.934, -109.25, 543.452, 1022.561, -3046.224, -3768.013, 8524.54,$$
$$6607.897, -12640.058, -5530.188, 9479.73, 1774, 639, -2821.526$$

- There are many large positive/negative values, such that a small change of features could lead to a significant change of output.

Ridge regression

- How to get smaller parameter values?
- We can assume that the parameter $\mathbf{w}$ (excluding the bias b) follow a zero-mean Gaussian prior

$$p(\mathbf{w}) = \mathcal{N}(\mathbf{w}|\mathbf{0}, \tau^2\mathbf{I}) \tag{9}$$

- For clarity, **we omit the bias w_0 in the following derivation.**
- Utilizing this prior, we obtain the maximum a posteriori (MAP) estimation

$$\mathbf{w}_{MAP} = \arg\max_{\mathbf{w}} \Big[\sum_{i=1}^{m} \log p(y_i|\mathbf{x}_i, \mathbf{w}) + \log p(\mathbf{w})\Big] \tag{10}$$

$$= \arg\max_{\mathbf{w}} \Big[\sum_{i=1}^{m} \log \mathcal{N}(\mathbf{x}_i^\top \mathbf{w}, \sigma^2) + \log \mathcal{N}(\mathbf{w}|\mathbf{0}, \tau^2\mathbf{I})\Big] \tag{11}$$

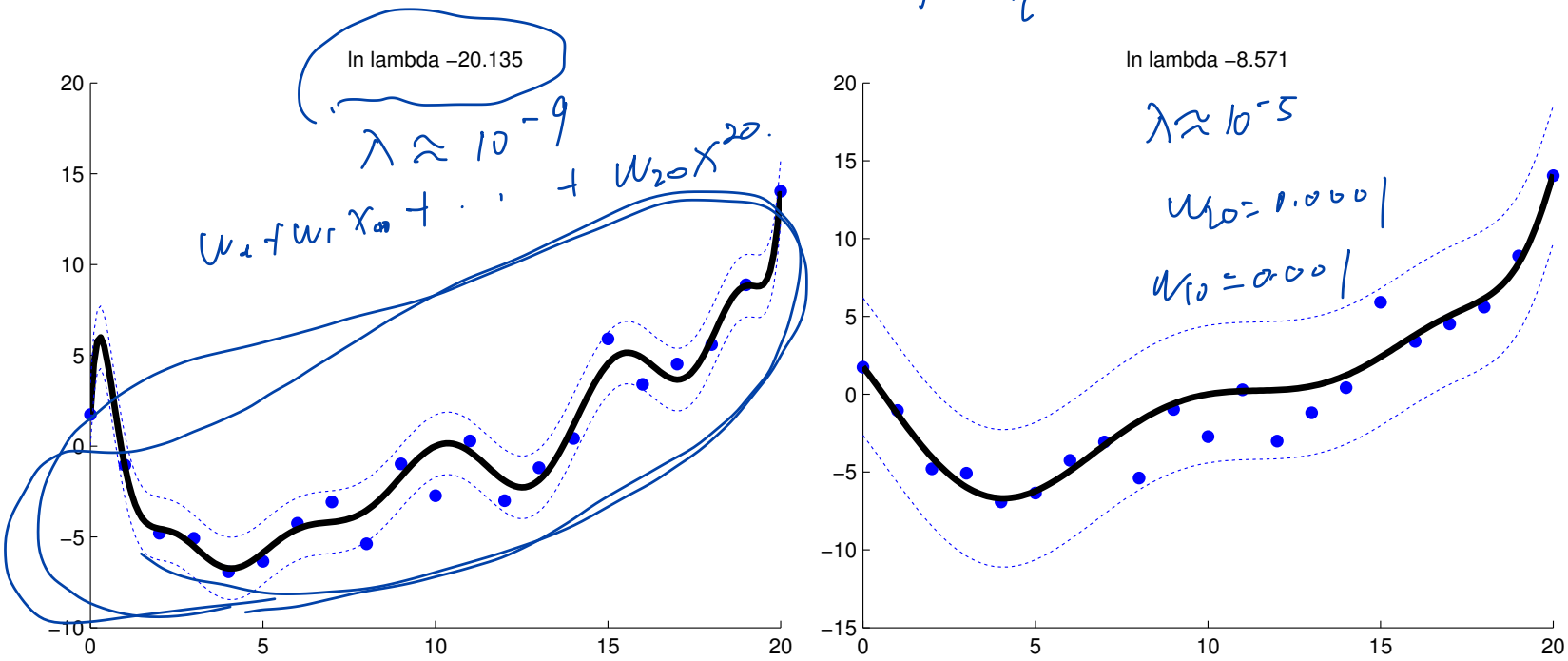
$$\equiv \arg\min_{\mathbf{w}} \Big[\sum_{i=1}^{m} (\mathbf{x}_i^\top \mathbf{w} - y_i)^2 + \lambda\|\mathbf{w}\|_2^2\Big]. \tag{12}$$

- The corresponding closed-form solution is given by

$$\mathbf{w}_{MAP} = (\lambda\mathbf{I} + \mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y}. \tag{13}$$

Ridge regression

- The above method is also known as ridge regression, or penalized least squares.
- In general, adding a Gaussian prior to the parameters of a model to encourage them to be small is called ℓ_2 **regularization** or **weight decay**.
- As shown below, when we set a larger λ, *i.e.*, more weight on the prior, the resulting curve will be smoother.

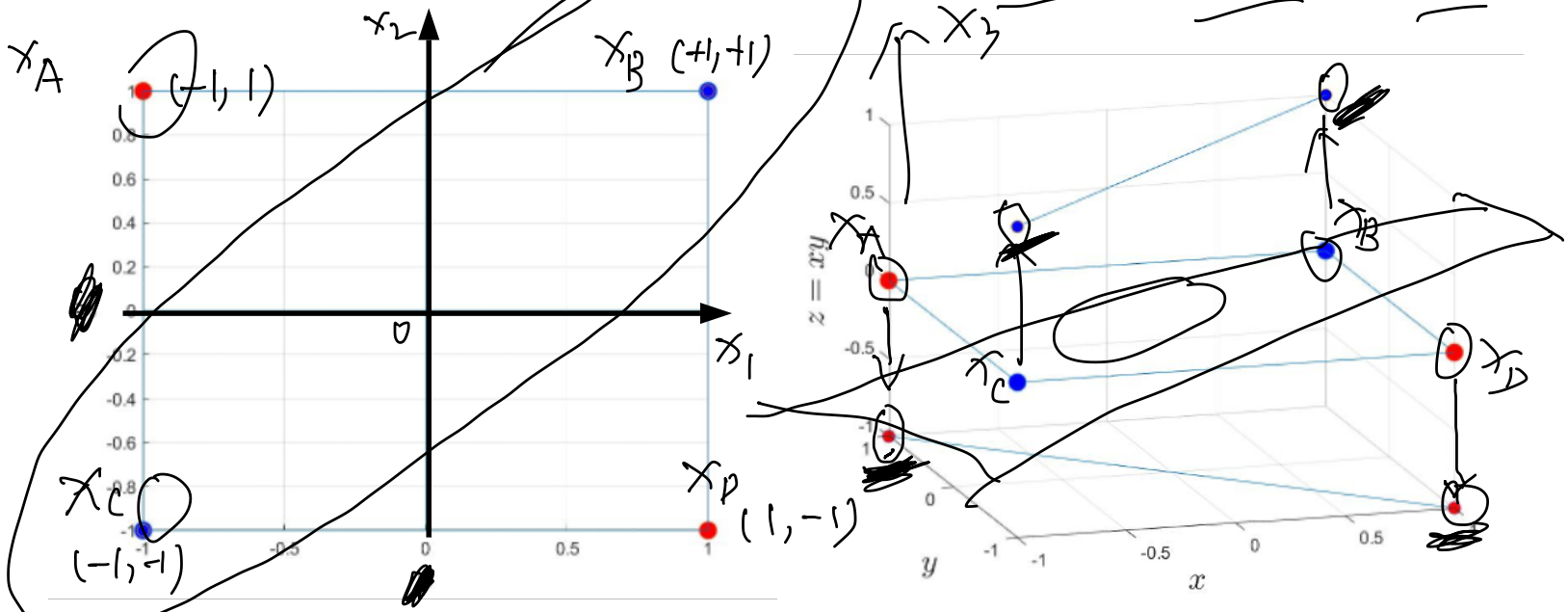


Polynomial regression

Motivation

- Some data may be not linearly separated, such as the classic **XOR** data, as shown on the bottom left.
- Consequently, the linear regression model doesn't work.
- To tackle it, we could project the original data to the **monomial** axis x_1x_2.
- Then, the XOR becomes linearly separated, as shown on the bottom right.
- Accordingly, we can design a novel linear regression model, as follows

$$f_{\mathbf{w}}(\mathbf{x}) = w_0 + w_1x_1 + w_2x_2 + w_{12}x_1x_2 + w_{11}x_1^2 + w_{22}x_2^2$$



Polynomial regression

Polynomial expansion

- The linear model $f_{\mathbf{w}}(\mathbf{x}) = \mathbf{x}^\top \mathbf{w}$ can be written as

$$f_{\mathbf{w}}(\mathbf{x}) = \sum_{i=0}^{d} x_i w_i = w_0 + \sum_{i=1}^{d} x_i w_i.$$

- By including terms involving the products of pairs of components of $\mathbf{x}$, we obtain a quadratic model:

$$f_{\mathbf{w}}(\mathbf{x}) = w_0 + \sum_{i=1}^{d} w_i x_i + \sum_{i=1}^{d} \sum_{j=1}^{d} w_{ij} x_i x_j.$$

Polynomial regression

$$C_{M}^{d+1} = \frac{(d+1)!}{(d+1-M)!\,M!} \simeq d^{M}$$

- In general:

$$f_{\mathbf{w}}(\mathbf{x}) = w_0 + \sum_{i=1}^{d} w_i x_i + \sum_{i=1}^{d}\sum_{j=1}^{d} w_{ij} x_i x_j + \sum_{i=1}^{d}\sum_{j=1}^{d}\sum_{k=1}^{d} w_{ijk} x_i x_j x_k + \ldots$$

Remarks

- For high dimensional d and high *polynomial order*, the number of polynomial terms becomes explosive! (In fact, this number of terms grows exponentially.)
- Hence, for high dimensional problems, polynomials of order larger than 3 are seldom used.

Polynomial regression

Linear model with basis expansion $\phi(\mathbf{x})$

$$f_{\mathbf{w}}(\mathbf{x}) = w_0 + \sum_{i=1}^{d} w_i x_i + \sum_{i=1}^{d}\sum_{j=1}^{d} w_{ij} x_i x_j + \sum_{i=1}^{d}\sum_{j=1}^{d}\sum_{k=1}^{d} w_{ijk} x_i x_j x_k + \dots$$
$$= \phi(\mathbf{x})^\top \mathbf{w},$$

where

$$\phi(\mathbf{x}) = [1, x_1, \dots, x_d, \dots, x_i x_j, \dots, x_i x_j x_k, \dots]^\top,$$
$$\mathbf{w} = [w_0, w_1, \dots, w_d, \dots, w_{ij}, \dots, w_{ijk}, \dots]^\top.$$

Note: $f_{\mathbf{w}}(\mathbf{x})$ is still a linear function *w.r.t.* $\mathbf{w}$, rather than $\mathbf{x}$. Thus, it is still a **linear model**.

Extending to the case of m data points, *i.e.*, $\mathbf{X} = [\mathbf{x}_1^\top; \dots; \mathbf{x}_m^\top] \in \mathbb{R}^{m\times(d+1)}$, the basis expansion is presented by

$$\mathbf{P}(\mathbf{X}) = [\phi(\mathbf{x}_1)^\top; \dots; \phi(\mathbf{x}_m)^\top] \in \mathbb{R}^{m\times|\mathbf{w}|}$$

Polynomial regression

Example

$\{\mathbf{x_i}, \mathbf{y_i}\}_{\mathbf{i=1}}^{\mathbf{m}}$ $\{x_1 = 0, x_2 = 0\} \to \{y = 0\}$
$\{x_1 = 1, x_2 = 1\} \to \{y = 1\}$
$\{x_1 = 1, x_2 = 0\} \to \{y = 2\}$
$\{x_1 = 0, x_2 = 1\} \to \{y = 3\}$

$2^{\mathbf{nd}}$ **order** polynomial model

$$f_{\mathbf{w}}(\mathbf{x}) = w_0 + w_1 x_1 + w_2 x_2 + w_{12} x_1 x_2 + w_{11} x_1^2 + w_{22} x_2^2$$

$$= \begin{bmatrix} 1 & x_1 & x_2 & x_1 x_2 & x_1^2 & x_2^2 \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ w_{12} \\ w_{11} \\ w_{22} \end{bmatrix}$$

4×6

$P^T P \to 6 \times 6$

$$\mathbf{P} = \begin{bmatrix} 1 & x_1 & x_2 & x_1 x_2 & x_1^2 & x_2^2 \\ 1 & x_1 & x_2 & x_1 x_2 & x_1^2 & x_2^2 \\ 1 & x_1 & x_2 & x_1 x_2 & x_1^2 & x_2^2 \\ 1 & x_1 & x_2 & x_1 x_2 & x_1^2 & x_2^2 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}$$

Polynomial regression

Ridge regression with original features $\mathbf{X}$:

$$\begin{aligned} \text{Learning:} \quad & \hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y} \\ \text{Prediction:} \quad & f_{\mathbf{w}}(\mathbf{X}_{new}) = \mathbf{X}_{new} \hat{\mathbf{w}} \end{aligned}$$

Ridge regression with basis expansion $\mathbf{P}(\mathbf{X})$:

$$\begin{aligned} \text{Learning:} \quad & \hat{\mathbf{w}} = (\mathbf{P}^\top \mathbf{P} + \lambda \mathbf{I})^{-1} \mathbf{P}^\top \mathbf{y} \\ \text{Prediction:} \quad & f_{\mathbf{w}}(\mathbf{P}(\mathbf{X}_{new})) = \mathbf{P}_{new} \hat{\mathbf{w}} \end{aligned}$$

$\phi(X_{new})$

Polynomial regression

For Regression Applications

- Learning: $\hat{\mathbf{w}} = (\mathbf{P}^\top \mathbf{P} + \lambda \mathbf{I})^{-1} \mathbf{P}^\top \mathbf{y}$, where $\mathbf{y}$ is continuous
- Prediction: $f_{\mathbf{w}}(\mathbf{P}(\mathbf{X}_{new})) = \mathbf{P}_{new} \hat{\mathbf{w}}$

For Classification Applications

- Learn **discrete** valued $\mathbf{y}$ (binary) or $\mathbf{Y}$ (one-hot)
- Binary Prediction: $f_{\mathbf{w}}(\mathbf{P}(\mathbf{X}_{new})) = \mathbf{sgn}(\mathbf{P}_{new} \hat{\mathbf{w}})$ if $y \in \{-1, +1\}$
- Multi-Category Prediction: $f_{\mathbf{w}}(\mathbf{P}(\mathbf{X}_{new})) = \mathbf{argmax}_{i=1,\ldots,C}(\mathbf{P}_{new} \hat{\mathbf{w}})$

Polynomial regression

Example (Cont'd)

$\{\mathbf{x_i}, \mathbf{y_i}\}_{\mathbf{i=1}}^{\mathbf{m}}$

$$\begin{aligned}
\{x_1 = 0, x_2 = 0\} &\rightarrow \{y = -1\} \\
\{x_1 = 1, x_2 = 1\} &\rightarrow \{y = -1\} \\
\{x_1 = 1, x_2 = 0\} &\rightarrow \{y = +1\} \\
\{x_1 = 0, x_2 = 1\} &\rightarrow \{y = +1\}
\end{aligned}$$

$2^{\mathbf{nd}}$ **order** polynomial model

$$\mathbf{P} = \begin{bmatrix} 1 & x_1 & x_2 & x_1x_2 & x_1^2 & x_2^2 \\ 1 & x_1 & x_2 & x_1x_2 & x_1^2 & x_2^2 \\ 1 & x_1 & x_2 & x_1x_2 & x_1^2 & x_2^2 \\ 1 & x_1 & x_2 & x_1x_2 & x_1^2 & x_2^2 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}$$

$$\hat{\mathbf{w}} = \mathbf{P}^\top (\mathbf{P}\mathbf{P}^\top)^{-1}\mathbf{y}$$ (note: under determined linear system)

$$= \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 6 & 3 & 3 \\ 1 & 3 & 3 & 1 \\ 1 & 3 & 1 & 3 \end{bmatrix}^{-1} \begin{bmatrix} -1 \\ -1 \\ +1 \\ +1 \end{bmatrix} = \begin{bmatrix} -1 \\ 1 \\ 1 \\ -4 \\ 1 \\ 1 \end{bmatrix}$$

Polynomial regression

Example (Cont'd)

Prediction:

Test point 1: $\{x_1 = 0.1, x_2 = 0.1\} \rightarrow \{y = \text{class } -1 \text{ or } +1?\}$

Test point 2: $\{x_1 = 0.9, x_2 = 0.9\} \rightarrow \{y = \text{class } -1 \text{ or } +1?\}$

Test point 3: $\{x_1 = 0.1, x_2 = 0.9\} \rightarrow \{y = \text{class } -1 \text{ or } +1?\}$

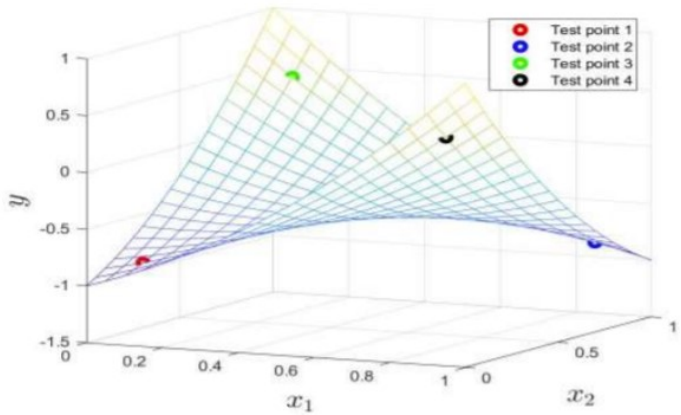
Test point 4: $\{x_1 = 0.9, x_2 = 0.1\} \rightarrow \{y = \text{class } -1 \text{ or } +1?\}$

$\hat{y} = \mathbf{P}_t \hat{\mathbf{w}}$

$$\hat{\mathbf{y}} = \mathbf{sgn}\left(\begin{bmatrix} 1 & 0.1 & 0.1 & 0.01 & 0.01 & 0.01 \\ 1 & 0.9 & 0.9 & 0.81 & 0.81 & 0.81 \\ 1 & 0.1 & 0.9 & 0.09 & 0.01 & 0.81 \\ 1 & 0.9 & 0.1 & 0.09 & 0.81 & 0.01 \end{bmatrix} \begin{bmatrix} -1 \\ 1 \\ 1 \\ -4 \\ 1 \\ 1 \end{bmatrix} \right)$$

$$= \text{sgn}\left(\begin{bmatrix} -0.82 \\ -0.82 \\ 0.46 \\ 0.46 \end{bmatrix} \right)$$

$$= \begin{bmatrix} -1 \\ -1 \\ 1 \\ 1 \end{bmatrix} \begin{array}{ll} \dashrightarrow & \text{Class } -1 \\ \dashrightarrow & \text{Class } -1 \\ \dashrightarrow & \text{Class } +1 \\ \dashrightarrow & \text{Class } +1 \end{array}$$



Lasso regression

Linear absolute shrinkage and selection operator

- We can replace the Gaussian prior by a Laplacian prior, *i.e.*,

$$p(\mathbf{w}) = \text{Lap}(\mathbf{w}|\mathbf{0}, \lambda) = \frac{1}{2\lambda} \exp\left(-\frac{\|\mathbf{w}\|_1}{\lambda}\right), \quad (14)$$

$= \sum_i |w_i|$

variance: $\sigma^2 = 2\lambda$

- The combination of the Gaussian distribution of $p(y|\mathbf{x}, \mathbf{w})$ and the Laplacian prior, leading to

MAP

$$\mathbf{w}_{MAP} = \arg\max_{\mathbf{w}} \left[\sum_{i=1}^{m} \log p(y_i|\mathbf{x}_i, \mathbf{w}) + \log p(\mathbf{w})\right] \quad (15)$$

$$= \arg\max_{\mathbf{w}} \left[\sum_{i=1}^{m} \log \mathcal{N}(\mathbf{w}^\top \mathbf{x}, \sigma^2) + \text{Lap}(\mathbf{w}|\mathbf{0}, b)\right] \quad (16)$$

log

$$\min_{\mathbf{w}} \frac{1}{2\sigma^2} \sum (y_i - x_i^T w)^2 + \frac{1}{\lambda} \|w\|_1$$

$$\equiv \arg\min_{\mathbf{w}} \left[\sum_{i=1}^{m} (\mathbf{x}_i^\top \mathbf{w} - y_i)^2 + \alpha \|\mathbf{w}\|_1\right]. \quad (17)$$

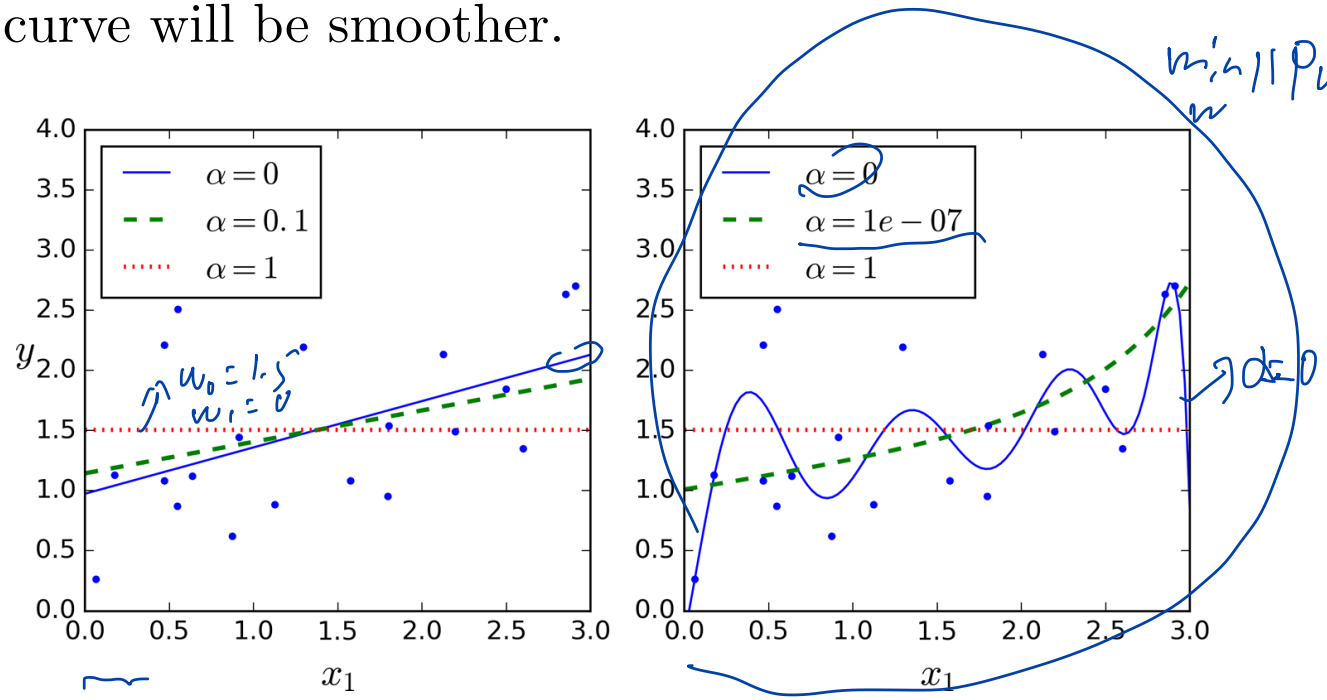
$$\min_{w} \sum (y_i - x_i^T w)^2 + \frac{2\sigma^2}{\lambda} \|w\|_1$$

$y = w^T x$

$= 0.1 x_1 + 0 \cdot x_2 + 0 \cdot x_3 + 0.2 x_4 + 0.5$

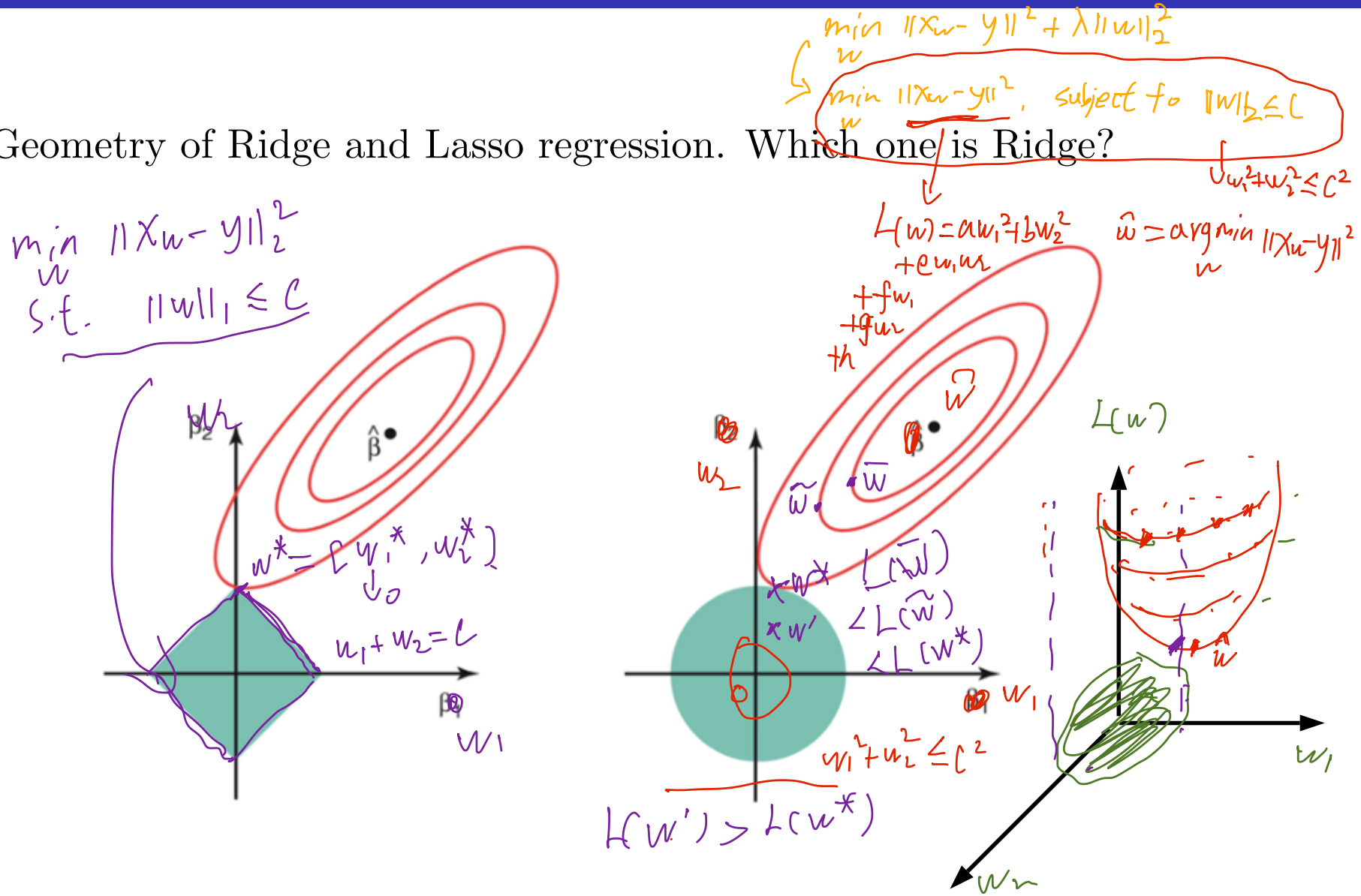
Lasso regression

- It is **Lasso regression**, and the regularization is called ℓ_1 **regularization**. It will encourage the sparse parameters.
- As shown below, when we set a larger α, *i.e.*, more weight on the prior, the resulting curve will be smoother.



Geometry of Ridge and Lasso regression

- Geometry of Ridge and Lasso regression. Which one is Ridge?

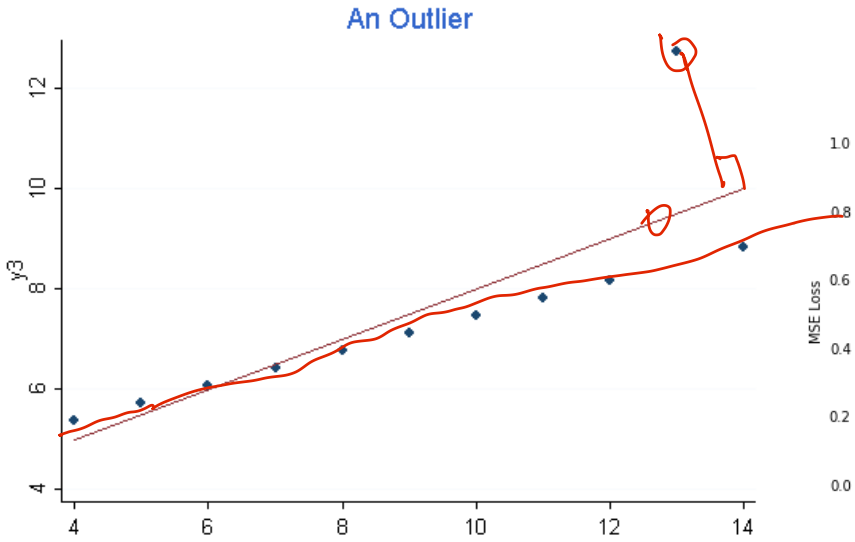


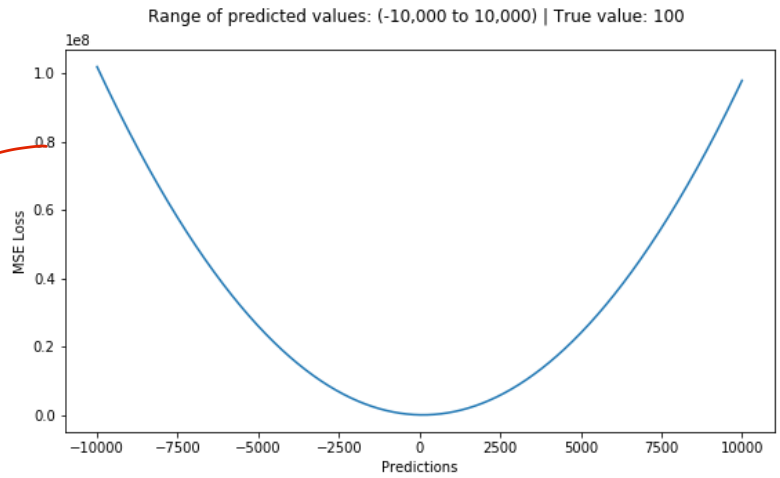
Robust linear regression

- When there is a few outliers in the training data D, which are far from most other points, then learned parameters $\mathbf{w}_{MLE}$ will be significantly influenced, leading to a very poor fitting.
- Let's see the loss curve of the residual sum of squares (RSS),

$$J(\mathbf{w}) = \frac{1}{2}\sum_{i=1}^{m}(\mathbf{x}_i^\top \mathbf{w} - y_i)^2. \tag{18}$$

- The error increases quadratically along with the residual. To minimize such a large error, the linear model will be significantly changed.
- How to alleviate the significant influence of outliers?



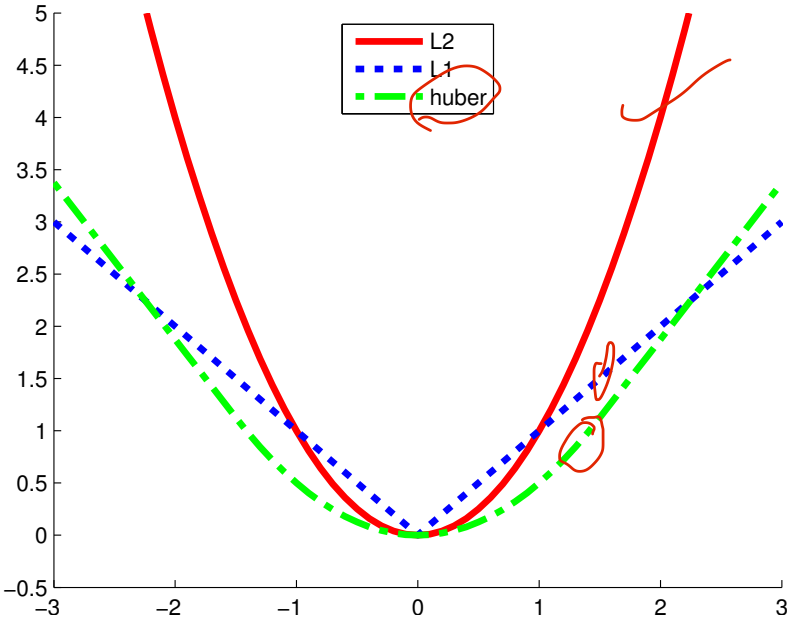


Robust linear regression

- We adopt the ℓ_1 loss to replace the ℓ_2 loss, as follows

$$J(\mathbf{w}) = \frac{1}{m}\sum_{i=1}^{m} |\mathbf{x}_i^\top \mathbf{w} - y_i|. \quad \text{MAE} \tag{19}$$

- The curves of ℓ_1 and ℓ_2 losses are shown as follows.
- When the residual is large, the ℓ_1 loss is much smaller than the ℓ_2 loss, such that the influence of outliers could be alleviated.



Robust linear regression

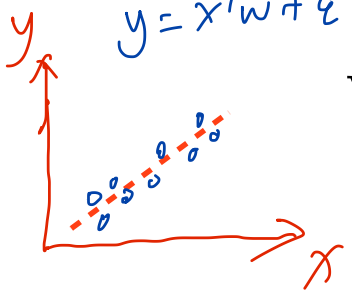
- Actually, the above ℓ_1 loss can also be derived from the probabilistic perspective, by assuming that

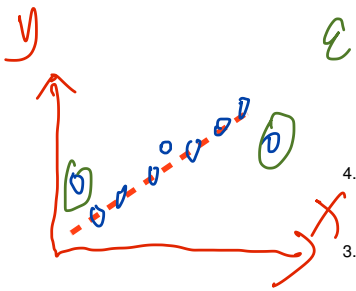
$$p(y|\mathbf{x}, \mathbf{w}, b) = \text{Lap}(y|\mathbf{w}^\top \mathbf{x}, b) \propto \exp(-\frac{1}{b}|y - \mathbf{w}^\top \mathbf{x}|) \tag{20}$$

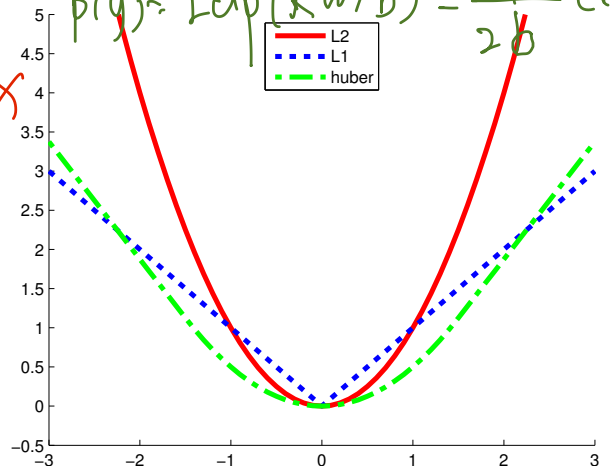
- Applying the maximum log-likelihood estimation (MLE), we will obtain

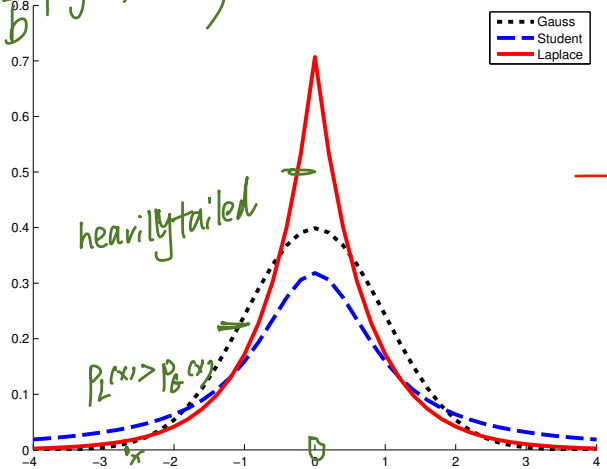
$$\mathbf{w}_{MLE} = \arg\max_{\mathbf{w}} \log \mathcal{L}(\mathbf{w}; D) = \arg\max_{\mathbf{w}} \sum_i^m \log p(y_i|\mathbf{x}_i, \mathbf{w}) \tag{21}$$

$$\equiv \arg\min_{\mathbf{w}} \frac{1}{b} \sum_{i=1}^m |\mathbf{x}_i^\top \mathbf{w} - y_i| \tag{22}$$









Robust linear regression

$$\mathbf{w}_{MLE} = \arg\min_{\mathbf{w}} \sum_{i=1}^{m} |\mathbf{x}_i^\top \mathbf{w} - y_i| \tag{23}$$

- However, the ℓ_1 loss function is non-differentiable. The gradient descent algorithm cannot be adopted.
- We can transform it to a linear program, as follows

$$\min_{\mathbf{w},\boldsymbol{t}} \sum_{i}^{m} t_i \tag{24}$$

$$s.t.\ -t_i \leq \mathbf{x}_i^\top \mathbf{w} - y_i \leq t_i, 1 \leq i \leq m.$$

Please refer to:
`https://math.stackexchange.com/questions/1639716/how-can-l-1-norm-minimization-with`

Robust linear regression

$$\mathbf{w}_{MLE} = \arg\min_{\mathbf{w}} \sum_{i=1}^{m} |\mathbf{x}_i^\top \mathbf{w} - y_i| \tag{25}$$

- We can also utilize the following equation:

$$|a| = \min_{\mu>0} \frac{1}{2}\left(\frac{a^2}{\mu} + \mu\right) \tag{26}$$

- Then, the above ℓ_1 minimization problem (25) can be reformulated as follows

$$\min_{\mathbf{w}} \min_{\mu_1,\ldots,\mu_m>0} \sum_{i=1}^{m} \frac{1}{2}\left(\frac{(\mathbf{x}_i^\top \mathbf{w} - y_i)^2}{\mu_i} + \mu_i\right). \tag{27}$$

- It can be iteratively and alternatively optimized as follows:
  - Given $\mathbf{w}$, $\mu_i = |\mathbf{x}_i^\top \mathbf{w} - y_i|, i = 1, \ldots, m$
  - Given μ, $\mathbf{w} = \arg\min_{\mathbf{w}} \sum_{i=1}^{m} \frac{1}{2}(\mathbf{x}_i^\top \mathbf{w} - y_i)^2/\mu_i$
- It is called iteratively reweighted least squares method.

Summary of different variants of linear regressions

Note that the uniform distribution will not change the mode of the likelihood. Thus, MAP estimation with a uniform prior corresponds to MLE.

$p(y\|\mathbf{x},\mathbf{w})$	$p(\mathbf{w})$	regression method
Gaussian	Uniform	Least squares
Gaussian	Gaussian	Ridge regression
Gaussian	Laplace	Lasso regression
Laplace	Uniform	Robust regression
Student	Uniform	Robust regression

